Fondamenti di Robotica

Luca Tortoriello

October 2025

Indice

1 La 'robotica' 4
- 1.1 Robotica Industriale . . . 4
- 1.2 Struttura meccanica dei robot . . . 5
  - 1.2.1 Robot manipolatori . . . 5
  - 1.2.2 Strutture robotiche principali . . . 7

2 Cinematica 10
- 2.1 Posa di un corpo rigido . . . 12
- 2.2 Matrice di rotazione . . . 13
  - 2.2.1 Matrici di rotazioni elementari . . . 13
- 2.3 Rappresentazione di un vettore . . . 14
- 2.4 Composizione di matrici di rotazione . . . 16
- 2.5 Angoli di Eulero (Rappresentazioni in forma minima) . . . 18
  - 2.5.1 Angoli ZYZ (terna corrente) . . . 19
  - 2.5.2 Angoli RPY (terna fissa) . . . 21
- 2.6 Asse-angolo . . . 23
- 2.7 Quaternione unitario . . . 25
- 2.8 Rototraslazioni (Trasformazioni omogenee) . . . 26
- 2.9 Cinematica diretta . . . 28
  - 2.9.1 Convenzione di Denavit-Hartenberg . . . 30

3 ROS 2 33
- 3.1 Ruolo di ROS nello sviluppo di sistemi robotici . . . 33
- 3.2 ROS 1 e ROS 2 . . . 34
- 3.3 Architettura di ROS 2 . . . 35
  - 3.3.1 Topics . . . 36
  - 3.3.2 Tipi di messaggi in ROS 2 . . . 37
  - 3.3.3 Services . . . 39
  - 3.3.4 Action . . . 40
- 3.4 Turtlesim . . . 41
  - 3.4.1 Topic in Turtlesim . . . 42
  - 3.4.2 Services in Turtlesim . . . 43
  - 3.4.3 Actions su Turtlesim . . . 43
- 3.5 Workspace . . . 44
  - 3.5.1 Build del Workspace . . . 44
- 3.6 I Pacchetti (Packages) . . . 45
- 3.7 I Nodi . . . 46
  - 3.7.1 Sviluppo di un nodo Python . . . 46
  - 3.7.2 Gestione e Debug dei Nodi . . . 48
- 3.8 Launch Files . . . 48
  - 3.8.1 Creazione di un pacchetto di Launch (Bringup) . . . 49
  - 3.8.2 Configurazione e Installazione . . . 49
  - 3.8.3 Esempio di Launch File in Python . . . 49
- 3.9 Modellazione Cinematica di un Robot . . . 50

3.9.1 Cinematica Diretta 50
3.9.2 Cinematica Inversa 50
3.9.3 Cinematica Differenziale 51
3.10 Modellazione Dinamica di un Robot 51
3.11 URDF (Unified Robot Description Format) 52
3.11.1 I Link (Collegamenti) 52
3.11.2 I Joint (Giunti) 52
3.12 Visualizzazione e Simulazione 53
3.12.1 RViz2 (ROS Visualization) 53
3.12.2 Gazebo: Il Simulatore Fisico 54
3.12.3 Gazebo Plugins e Integrazione ROS 2 54
3.12.4 Messaggi Interni Gazebo (gz msgs) 54
3.13 ArUCO Marker 54
3.13.1 Algoritmo di Decodifica 56
3.13.2 Calibrazione della Camera e Stima della Posa 57

1 La 'robotica'

Il termine **robot** fu introdotto dal commediografo ceco Karel Čapek nel 1920 nel dramma *R.U.R. – Rossum's Universal Robots*. In quest'opera l'autore coniò il termine a partire dalla parola ceca **robota**, utilizzata nelle lingue slave per indicare il lavoro forzato o esecutivo. Nel contesto della rappresentazione teatrale, il termine veniva utilizzato per indicare gli **automi artificiali** costruiti da Rossum, progettati per svolgere lavori al posto degli esseri umani. Nella trama, tuttavia, queste creature finiscono per ribellarsi ai loro creatori, dando origine a una riflessione sul rapporto tra uomo, lavoro e tecnologia. L'immagine di robot come artefatto meccanico comincia negli anni '40, allorché il russo Isaac Asimov, concepì il robot come un automa di sembianze umane, ma privo di sentimento.

Il termine **robotica** fu così introdotto da Asimov per indicare la scienza devota allo studio dei robot, fondata sull'osservanza di **tre leggi fondamentali**:

1. Un robot non può far del male a un essere umano ne consentire, restando inoperoso, che un essere umano si trovi in pericolo
2. Un robot deve obbedire agli ordini impartiti da esseri umani, a meno che tali ordini non entrino in conflitto con la prima legge
3. Un robot deve proteggere la sua esistenza a meno che tale protezione non vada in conflitto con la prima o la seconda legge

Si nota che l'elemento chiave in queste tre leggi sia la **sicurezza nell'interazione con le persone**. I robot come robot industriali, sono regolamentati da una normativa dedicata, lo standard `ISO 8373` che ne da anche una definizione:

> Manipolatore controllato automaticamente, riprogrammabile e multifunzionale, che si può muovere in tre o più assi

Si considerano quindi strutture tridimensionali, che si possano muovere nello spazio, che devono svolgere più di una funzione, che attraverso la parte software riescono a svolgere delle funzioni con algoritmi programmabili.

1.1 Robotica Industriale

La robotica industriale è la disciplina che si occupa della progettazione, dello sviluppo e dell'impiego dei robot in ambito industriale. La principale differenza tra il contesto industriale tradizionale e altri contesti applicativi riguarda il tipo di ambiente operativo: nel mondo industriale, poiché le operazioni da svolgere sono generalmente note a priori, l'**ambiente** è tipicamente **strutturato** e quindi più prevedibile.

Un robot è un sistema complesso che integra diverse componenti hardware e software. L'architettura di un manipolatore robotico può essere descritta attraverso i seguenti blocchi funzionali:

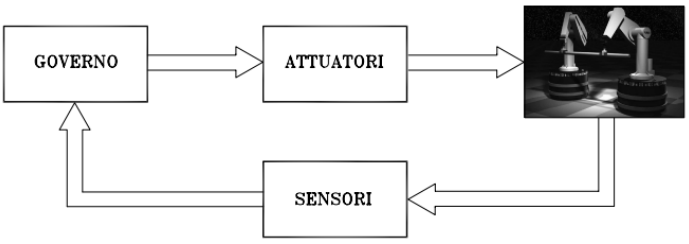


Figura 1.1: Componenti di un sistema robotico

- **Attuatori**: dispositivi responsabili della generazione del movimento. Sono dotati di meccanismi di attuazione che trasformano l'energia fornita da una sorgente esterna in movimento meccanico. La struttura meccanica alla quale viene trasmesso il movimento prende il nome di **meccanismo**.
- **Sensori**: permettono di ottenere informazioni sullo stato del robot e sull'ambiente circostante. In particolare, i *sensori propriocettivi* forniscono informazioni sullo stato interno della macchina, ad esempio in termini di **posizione** o velocità delle articolazioni, mentre i *sensori esterocettivi* consentono di acquisire informazioni relative all'ambiente esterno
- **Governo**: rappresenta il sistema di controllo che **integra percezione e azione**. Ha il compito di comandare l'esecuzione delle azioni in funzione degli obiettivi stabiliti da una tecnica di **pianificazione** del compito, tenendo conto dei vincoli imposti sia dalla struttura del robot sia dall'ambiente operativo

1.2 Struttura meccanica dei robot

I robot possono distinguersi in quelli a base fissa (*robot manipolatori*) e quelli a base mobile (*robot mobili*). Nel seguito vengono illustrate le caratteristiche geometriche delle due classi.

1.2.1 Robot manipolatori

La struttura meccanica di un robot manipolatore è costituita da un insieme di corpi rigidi, detti **bracci**, collegati tra loro tramite **articolazioni** o **giunti**. In generale, un manipolatore è composto da tre elementi principali: una **struttura portante**, che garantisce la mobilità del sistema, un **polso**, che consente di orientare l'organo terminale, e un **organo terminale** (end-effector), responsabile dell'esecuzione del compito assegnato al robot.

Dal punto di vista strutturale, i manipolatori sono generalmente modellati come **catene cinematiche**. Nella maggior parte dei casi si tratta di una **catena cinematica aperta**, i bracci sono collegati in sequenza formando una struttura che unisce due estremi senza creare cicli. Alcuni sistemi possono presentare una

catena cinematica chiusa, nella quale i collegamenti tra i bracci formano un anello.

La **mobilità** del manipolatore è garantita dalla presenza dei giunti. Tra due bracci consecutivi può essere realizzato un **giunto prismatico** oppure un **giunto rotoidale**. In una catena cinematica aperta, ciascun giunto introduce un singolo **grado di libertà**. In particolare, un giunto prismatico consente un movimento relativo di **traslazione** tra due bracci, mentre un giunto rotoidale permette un movimento relativo di **rotazione**. Nella pratica, i giunti rotoidali sono spesso preferiti per la loro maggiore compattezza e affidabilità. Nel caso di catene cinematiche chiuse, invece, il numero effettivo di gradi di libertà risulta inferiore al numero di giunti a causa dei vincoli introdotti dalla struttura ad anello. Utilizzando e combinando giunti elementari si possono ricavare anche movimenti più complessi (es. *giunto sferico*).

I **gradi di libertà** devono essere distribuiti lungo la struttura del manipolatore in modo da garantire la capacità di eseguire il compito richiesto. Nel caso generale di manipolazione nello spazio tridimensionale, per posizionare e orientare arbitrariamente un oggetto sono necessari **sei gradi di libertà**: tre per definire la posizione nello spazio e tre per determinarne l'orientamento rispetto a un sistema di riferimento. Se il manipolatore dispone di un numero di gradi di libertà superiore a quello richiesto dal compito, esso viene definito **ridondante** dal punto di vista cinematico.

Un concetto importante nella progettazione dei manipolatori è quello di **spazio di lavoro** (*workspace*), il luogo dei punti che possono essere raggiunti nello spazio dal manipolatore effettuando tutti i moti possibili. La forma e l'estensione di tale spazio dipendono dalla configurazione della struttura meccanica e dai **limiti di corsa** dei giunti. All'interno del *workspace* possiamo distinguere due ulteriori sottospazi:

1. **Spazio di lavoro destro**: il luogo dei punti raggiungibili con più di un orientamento
2. **Spazio di lavoro raggiungibile**: il luogo dei punti raggiungibili con un unico orientamento

Nello studio del manipolatore si tende a dividere la catena cinematica in compartimenti:

- La **base** (struttura portante): parte del manipolatore che definisce la posizione dello spazio. Nella numerazione corrisponde al link 0
- Il **polso**: definisce l'orientamento

La struttura portante del manipolatore ha quindi il compito di posizionare il polso del robot nello spazio, mentre il polso è responsabile dell'orientamento dell'organo terminale. In uno spazio tridimensionale, sono quindi necessari almeno tre gradi di libertà per il posizionamento del polso e tre per l'orientamento dell'end-effector. In base alla tipologia e alla successione dei giunti presenti nella

struttura, a partire dall'articolazione alla base, è possibile classificare i manipolatori in diverse categorie, tra cui **cartesiani**, **cilindrici**, **sferici**, **SCARAS** e **antropomorfi**.

Nei datasheet spesso sono fornite come informazioni:

- **L'accuratezza**: la deviazione (errore) tra la posizione raggiunta in un postura assegnata e la posizione che viene calcolata attraverso i dati di targa del manipolatore, mediante la *cinematica diretta* ($<$ 1mm)
- **La ripetibilità**: fornisce una misura della capacità del manipolatore di ritornare in una posizione precedentemente raggiunta ($<$ 0.2 mm)

1.2.2 Strutture robotiche principali

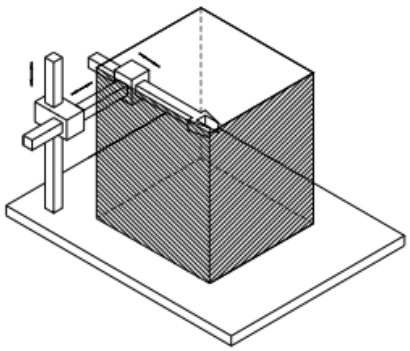


Figura 1.2: Manipolatore cartesiano e suo spazio di lavoro

La geometria cartesiana è realizzata con tre giunti prismatici con assi ugualmente ortogonali tra di loro.

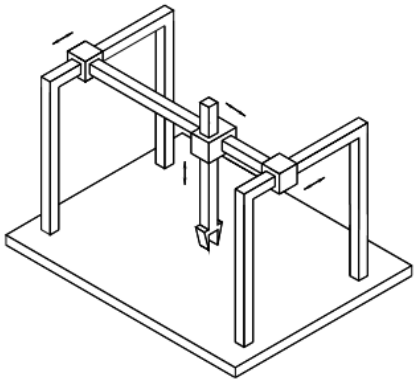


Figura 1.3: Manipolatore a portale

Se si desidera un approccio dall'alto, il manipolatore cartesiano può essere realizzato con una struttura *a portale (gantry)*, che consente di ottenere uno spazio di lavoro di grosso volume e di manipolare oggetti di peso e dimensione rilevanti.

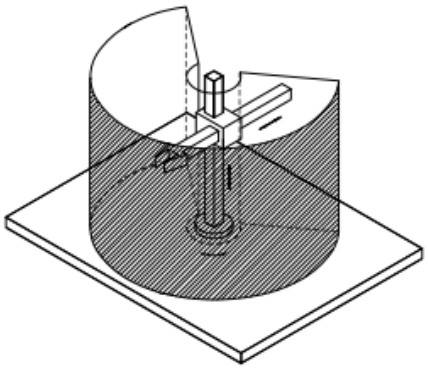


Figura 1.4: Manipolatore cilindrico e suo spazio di lavoro

La geometria cilindrica si differenzia da quella cartesiana per la sostituzione del giunto prismatico di base con uno rotoidale.

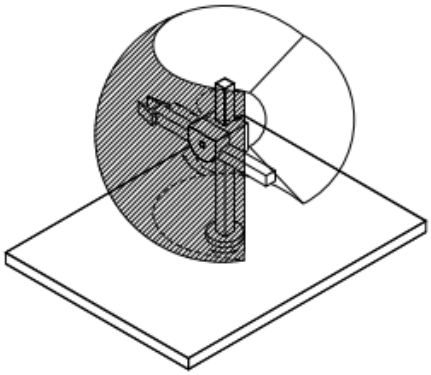


Figura 1.5: Manipolatore sferico e suo spazio di lavoro

La geometria sferica si differenzia da quella cilindrica per la sostituzione del secondo giunto (prismatico) con un giunto rotoidale.

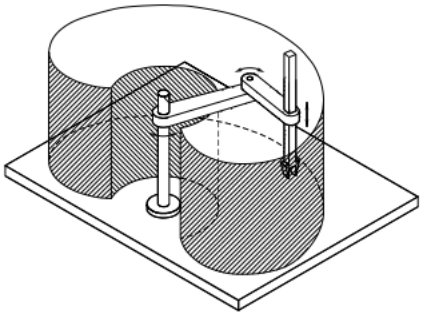


Figura 1.6: Manipolatore SCARA e suo spazio di lavoro

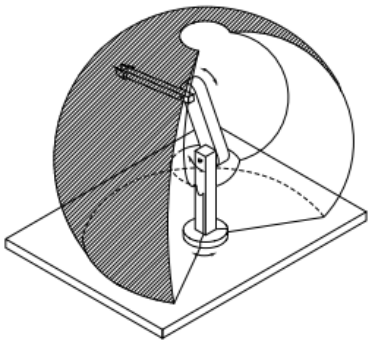


Figura 1.7: Manipolatore antropomorfo e suo spazio di lavoro

2 Cinematica

La cinematica dei manipolatori studia le relazioni, limitarmente al movimento, tra lo **spazio dei giunti**, in cui definisco il moto di tutti i giunti, e lo spazio operativo con cui viene descritto il moto dell'**organo terminale** o la pinza. Per poter studiare la relazione tra i due spazi si introducono delle **convenzioni**:

1. Tutti i giunti rotoidali sono rappresentati con dei cilindri e l'asse principale del cilindro è l'asse di rotazione
2. Si assume che l'angolo sia positivo quando la rotazione avviene in senso antiorario
3. Ai giunti rotoidali si associa simbolicamente il simbolo θ. La numerazione parte da 1
4. Ai giunti prismatici sono associati dei cubi. Il simbolo di variabile di giunto prismatico è la d
5. Per poter specificare il moto deve essere introdotto un **sistema di riferimento**. La numerazione delle terne z associate parte da 0

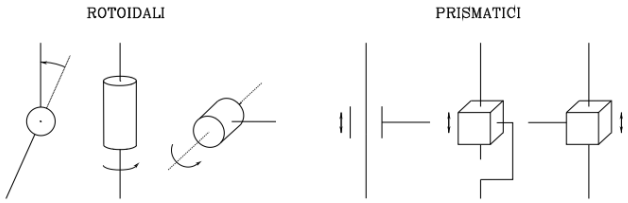


Figura 2.1: Rappresentazioni convenzionali dei giunti

Nella numerazione delle terne si parte dalla base, si associa una terna di giunto e l'ultima terna è collocata in corrispondenza dell'**organo terminale**. L'introduzione delle terne consente di studiare le relazioni di rototraslazione tra una terna successiva. L'insieme di tutte le **rototraslazioni** consente di descrivere la **cinematica diretta** del manipolatore. Per diretta si intende che si è in grado di capire, noto il moto di tutti i giunti, qual è la **posizione finale dell'organo terminale**.

L'**insieme delle variabili di giunto** θ_i e d_i è raggruppato in un unico vettore q di cardinalità n. Lo spazio operativo, chiamato **spazio cartesiano**, è definito da un vettore di **posa**:

$$x = (p, \phi)^\top , \tag{2.1}$$

che individua la posizione p ed orientamento ϕ dell'organo terminale. Nel caso si riescano ad esplicare tutte le variabili di posizione e orientamento, x ha dimensione 6. La **dimensione** dello **spazio operativo** si indica con la lettera m.

Se:

$$m > n\,,$$

il manipolatore è ridondante.

I manipolatori sono sistemi soggetti a **vincoli**. Il sistema, muovendosi nel tempo, ammette della variabili dipendenti da esso:

$$\begin{pmatrix} q_1 \\ \vdots \\ q_n \end{pmatrix} = \begin{pmatrix} q_1(t) \\ \vdots \\ q_n(t) \end{pmatrix}$$

La velocità e l'accelerazione, di conseguenza, si esprimono come le derivate temporali del vettore dei giunti:

$$\dot{q}_i(t) \quad \text{velocità}$$

$$\ddot{q}_i(t) \quad \text{accelerazione}$$

Si distinguono tre principali tipologie di vincoli:

1. **Vincoli olonomi** (*vincoli di posizione*)
   Sono vincoli esprimibili tramite relazioni funzionali tra le coordinate generalizzate del sistema, eventualmente dipendenti anche dal tempo:

   $$f(q_1, q_2, \ldots, q_n, t) = 0$$

   In particolare:

   - Se il vincolo è espresso da un'uguaglianza ($= 0$), si parla di **vincolo bilaterale**
   - Se il vincolo è espresso da una disuguaglianza (≥ 0 o ≤ 0), si parla di **vincolo unilaterale**

2. **Vincoli anolonomi** (*vincoli di mobilità*)
   Sono vincoli che non possono essere espressi unicamente come relazioni tra le coordinate, ma coinvolgono anche le velocità (e talvolta il tempo). In forma generale:

   $$g(q_1, \ldots, q_n, \dot{q}_1, \ldots, \dot{q}_n, t) = 0$$

   oppure in forma differenziale non integrabile:

   $$\sum_{i=1}^{n} a_i(q,t)\,\dot{q}_i + a_0(q,t) = 0$$

3. **Vincolo di corpo rigido**
   Un corpo R si dice *rigido* se la distanza tra qualunque coppia di suoi punti A e B rimane costante nel tempo:

   $$\|\mathbf{r}_A - \mathbf{r}_B\| = \text{costante} \quad \forall\, t$$

   Questo vincolo implica che il corpo non subisce deformazioni durante il moto

2.1 Posa di un corpo rigido

Un corpo rigido nello spazio è completamente descritto dalla sua **posizione** e dal suo **orientamento** rispetto ad una terna di riferimento. L'insieme di queste due informazioni prende il nome di *posa* del corpo rigido. In riferimento alla Figura 2.2:

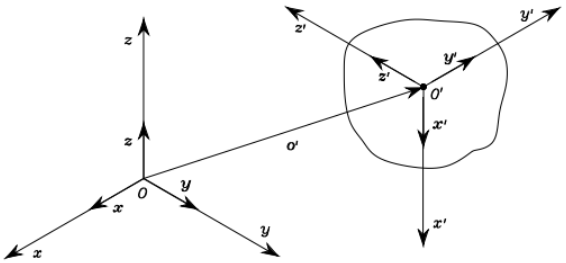


Figura 2.2: Posizione e orientamento di un corpo rigido

Sia $O - xyz$ una terna ortonormale di riferimento e siano x, y e z i versori dei suoi assi. La posizione di un punto O' appartenente al corpo rigido rispetto alla terna $O - xyz$ può essere espressa come combinazione lineare dei versori della terna di riferimento:

$$o' = o'_x x + o'_y y + o'_z z,$$

dove o'_x, o'_y e o'_z rappresentano le componenti del vettore $o' \in \mathbb{R}^3$ lungo gli assi della terna di riferimento. La posizione del punto O' può quindi essere rappresentata tramite il vettore colonna (3×1):

$$O' = \begin{bmatrix} O'_x \\ O'_y \\ O'_z \end{bmatrix}$$

Il vettore O' è un vettore applicato poiché, oltre a essere caratterizzato da direzione e modulo, è anche definito il suo punto di applicazione.

Per descrivere l'orientamento del corpo rigido è utile introdurre una seconda terna ortonormale solidale al corpo e riferirne i versori alla terna di riferimento. Si consideri quindi la terna $O' - x'y'z'$ con origine nel punto O' e siano x', y' e z' i versori dei suoi assi. Rispetto alla terna di riferimento $O - xyz$, tali versori possono essere espressi come:

$$\begin{aligned} x' &= x'_x x + x'_y y + x'_z z \\ y' &= y'_x x + y'_y y + y'_z z \\ z' &= z'_x x + z'_y y + z'_z z \end{aligned} \tag{2.2}$$

Le componenti di ciascun versore rappresentano i **coseni direttori** degli assi della terna solidale $O' - x'y'z'$ rispetto agli assi della terna di riferimento $O - xyz$.

Il vettore $\overline{OO'}$ rappresenta la **traslazione tra le origini** delle due terne e quindi definisce la distanza tra i loro centri.

2.2 Matrice di rotazione

L'equazione (2.2) si può scrivere in forma compatta. I tre versori che esprimono l'orientamento possono essere combinati nella matrice (3×3):

$$R = \begin{bmatrix} x' & y' & z' \end{bmatrix} = \begin{bmatrix} x'_x & y'_x & z'_x \\ x'_y & y'_y & z'_y \\ x'_z & y'_z & z'_z \end{bmatrix} = \begin{bmatrix} x'^{\top}x & y'^{\top}x & z'^{\top}x \\ x'^{\top}y & y'^{\top}y & z'^{\top}y \\ x'^{\top}z & y'^{\top}z & z'^{\top}z \end{bmatrix}, \qquad (2.3)$$

che prende il nome di **matrice di rotazione**. R esprime la **rotazione** della terna primo rispetto alla terna base, composta dalle componenti dei versori della terna primo rispetto alla terna base. È significativo notare che i vettori colonna della matrice R, essendo i versori di una terna ortonormale, sono ortogonali fra di loro

$$x'^{\top}y' = 0 \quad y'^{\top}z' = 0 \quad z'^{\top}x' = 0$$

e, inoltre, hanno modulo unitario:

$$x'^{\top}x' = 1 \quad y'^{\top}y' = 1 \quad z'^{\top}z' = 1$$

Di conseguenza, la matrice R risulta **ortogonale**, ovvero gode della proprietà

$$R^{\top}R = I_3$$

ove I_3 indica la matrice identità di dimensioni 3×3.

Se si moltiplicano da destra ambo i membri della per la matrice inversa R^{-1}, si ha il risultato notevole

$$R^{\top} = R^{-1}, \qquad (2.4)$$

ovvero la trasposta della matrice di rotazione coincide con la sua inversa.

2.2.1 Matrici di rotazioni elementari

Una rotazione è elementare se avviene attorno ad un singolo asse coordinato. Si abbia la rotazione della terna di riferimento $O - xyz$ di un angolo α intorno all'asse z (Figura 2.3), e sia $O' - x'y'z'$ la nuova terna ottenuta.

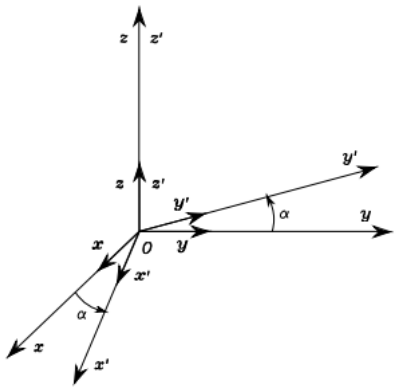


Figura 2.3: Rotazione della terna $O - xyz$ di un angolo α, attorno all'asse z

I versori degli assi della nuova terna possono essere caratterizzati in termini delle componenti rispetto agli assi della terna di riferimento, ovvero

$$x' = \begin{bmatrix} \cos\alpha \\ \sin\alpha \\ 0 \end{bmatrix}, \quad y' = \begin{bmatrix} -\sin\alpha \\ \cos\alpha \\ 0 \end{bmatrix}, \quad z' = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix},$$

per cui la matrice di rotazione della terna $O - x'y'z'$ rispetto alla terna $O - xyz$ risulta

$$R_z(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{2.5}$$

In maniera analoga si trova che le rotazioni di un angolo β intorno all'asse y e di un angolo γ intorno all'asse x risultano, rispettivamente,

$$R_y(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix} \tag{2.6}$$

$$R_x(\gamma) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\gamma & -\sin\gamma \\ 0 & \sin\gamma & \cos\gamma \end{bmatrix} \tag{2.7}$$

2.3 Rappresentazione di un vettore

Si consideri il caso in Figura 2.4 in cui l'origine della terna solidale al corpo coincide con l'origine della terna di riferimento; risulta $O' = \underline{0}$, ove $\underline{0}$ denota il vettore (3×1) di elementi tutti nulli.

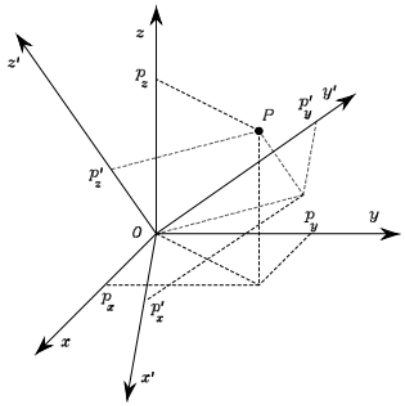


Figura 2.4: Rappresentazione di un punto P in terne differenti di coordinate

Un punto P nello spazio può essere rappresentato nelle due terne di coordinate. Sia:

$$p = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} ,$$

il vettore rappresentativo delle coordinate di P espresse nella terna $O - xyz$, e:

$$p' = \begin{bmatrix} p'_x \\ p'_y \\ p'_z \end{bmatrix}$$

il vettore rappresentativo delle coordinate di P espresse nella terna $O - x'y'z'$.

Poiché p e p' sono rappresentazioni dello stesso punto P si ha:

$$p = p'_x x' + p'_y y' + p'_z z' = \begin{bmatrix} x' & y' & z' \end{bmatrix} p'$$

che equivale a scrivere:

$$p = Rp' \tag{2.8}$$

In virtù della proprietà di ortogonalità, si ha che la trasformazione inversa è caratterizzata semplicemente da

$$p' = R^\top p$$

Una matrice di rotazione assume pertanto tre **significati geometrici equivalenti**:

- fornisce l'orientamento di una terna di coordinate rispetto a un'altra: i vettori colonna sono i coseni direttori degli assi della terna ruotata rispetto alla terna di partenza

- rappresenta una trasformazione di coordinate che mette in relazione le coordinate di uno stesso punto in due terne differenti (di origine comune)
- è l'operatore che consente di ruotare un vettore in una stessa terna di coordinate

2.4 Composizione di matrici di rotazione

Per ricavare le regole di composizione delle matrici di rotazione è utile considerare l'espressione di uno stesso vettore rispetto a diverse terne di riferimento. Siano assegnate tre terne di coordinate $O{-}x_0y_0z_0$, $O{-}x_1y_1z_1$ e $O{-}x_2y_2z_2$ aventi origine comune O. Il vettore p, che individua la posizione di un generico punto nello spazio, può essere espresso in ciascuna delle tre terne; siano rispettivamente p^0, p^1 e p^2 le coordinate di p nelle diverse terne.

La relazione tra le coordinate del vettore espresse nella terna 2 e nella terna 1 è definita dalla matrice di rotazione R_2^1, che descrive l'orientamento della terna 2 rispetto alla terna 1:

$$p^1 = R_2^1\, p^2 \tag{2.9}$$

Analogamente, per le altre trasformazioni tra sistemi di riferimento si ha:

$$p^0 = R_1^0\, p^1, \tag{2.10}$$

$$p^0 = R_2^0\, p^2. \tag{2.11}$$

Sostituendo (2.9) in (2.10) e confrontando con (2.11), si ottiene la **proprietà di composizione** delle matrici di rotazione:

$$R_2^0 = R_1^0\, R_2^1 \tag{2.12}$$

Dal punto di vista geometrico, la rotazione complessiva R_2^0 equivale alla successione di due rotazioni: prima dalla terna 2 alla terna 1 (descritta da R_2^1), e successivamente dalla terna 1 alla terna 0 (descritta da R_1^0).

La matrice R_2^0 può essere ottenuta secondo due interpretazioni differenti, a seconda del sistema di riferimento rispetto al quale vengono espresse le rotazioni successive:

1. **Rispetto alla terna corrente (assi mobili)** Ogni rotazione è espressa rispetto alla terna ottenuta al passo precedente; le terne si muovono solidalmente con il corpo. La regola (2.12) si applica direttamente: le matrici si premoltiplicano da sinistra verso destra nell'ordine della sequenza di rotazioni:
$$R_2^0 = R_1^0\, R_2^1 \tag{2.13}$$

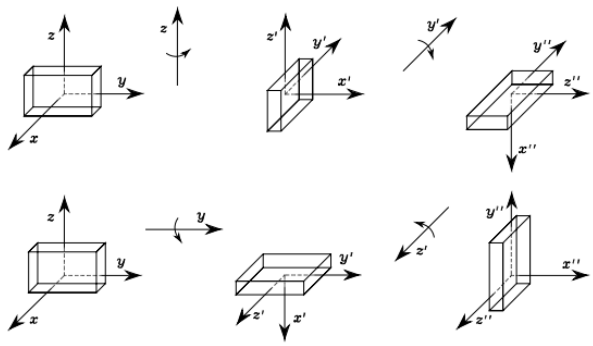


Figura 2.5: Rotazioni successive di un solido intorno ad assi della terna corrente.

2. **Rispetto alla terna fissa (assi costanti)** Tutte le rotazioni sono effettuate rispetto agli assi della terna iniziale $O{-}x_0y_0z_0$, che rimane fissa. Per ricavare la matrice risultante $\bar{R}_2^0$, si procede come segue:

   - Si riallinea la terna 1 con la terna 0 mediante la rotazione R_0^1
   - Si effettua la rotazione richiesta $\bar{R}_2^1$ rispetto alla terna corrente (ora coincidente con la terna 0)
   - Si compensa il riallineamento iniziale applicando la rotazione inversa R_1^0

   Applicando la regola di composizione (2.12) a questa sequenza si ottiene:

   $$\bar{R}_2^0 = R_1^0\, R_0^1\, \bar{R}_2^1\, R_1^0 \tag{2.14}$$

   Poiché $R_1^0\, R_0^1 = I$ (la rotazione e la sua inversa si annullano):

   $$\bar{R}_2^0 = \bar{R}_2^1\, R_1^0 \tag{2.15}$$

   In generale, per n rotazioni successive rispetto alla terna fissa si ha:

   $$\bar{R}_n^0 = \bar{R}_n^{n-1} \cdots R_2^1\, R_1^0 \tag{2.16}$$

   Le matrici si postmoltiplicano da destra verso sinistra nell'ordine della sequenza di rotazioni.

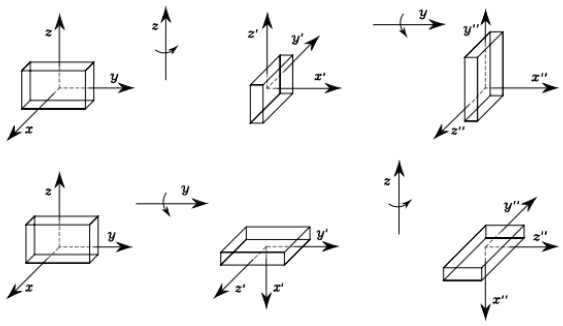


Figura 2.6: Rotazioni successive di un solido intorno ad assi della terna fissa.

Poiché le terne considerate sono ortonormali, la matrice inversa di una matrice di rotazione coincide con la sua trasposta:

$$R_i^j = \left(R_j^i\right)^{-1} = \left(R_j^i\right)^\top \tag{2.17}$$

È importante osservare che il prodotto tra matrici di rotazione **non è commutativo**: l'ordine con cui vengono applicate le rotazioni influenza il risultato finale, e la composizione di più rotazioni dipende dalla sequenza con cui le singole rotazioni si succedono.

2.5 Angoli di Eulero (Rappresentazioni in forma minima)

Le matrici di rotazione costituiscono uno strumento fondamentale per la descrizione dell'orientamento di una terna nello spazio tridimensionale. Tuttavia, tale rappresentazione risulta **ridondante**: una matrice di rotazione è composta da nove elementi, che rappresentano le coordinate dei versori della terna mobile espressi rispetto alla terna di riferimento. Poiché tali vettori sono versori, essi devono soddisfare due classi di vincoli:

- **Tre condizioni di normalizzazione**, che impongono la norma unitaria di ciascun versore
- **Tre condizioni di ortogonalità**, che garantiscono la mutua perpendicolarità tra di essi

Le nove componenti della matrice non sono quindi indipendenti, bensì soggette a sei vincoli complessivi. Il numero di parametri indipendenti necessari a descrivere un orientamento nello spazio tridimensionale si riduce pertanto a:

$$9 - 6 = 3 \tag{2.18}$$

Una rappresentazione dell'orientamento in termini di tre parametri indipendenti è detta **rappresentazione minima**. La più nota è quella basata sugli **angoli**

di Eulero, in cui l'orientamento è descritto da una tripletta di angoli:

$$\Phi = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}, \tag{2.19}$$

ciascuno dei quali rappresenta una rotazione elementare attorno a uno degli assi coordinati.

Considerando tutte le possibili sequenze di tre rotazioni elementari attorno agli assi di una terna $O{-}xyz$, si ottengono in linea di principio $3^3 = 27$ combinazioni. Tuttavia, è necessario escludere le sequenze in cui due rotazioni consecutive avvengono attorno al medesimo asse: in tal caso, infatti, i due angoli non sarebbero distinguibili singolarmente, ma soltanto attraverso la loro somma, compromettendo l'unicità della rappresentazione. Eliminando tali combinazioni degeneri, si ottengono **12 rappresentazioni valide** degli angoli di Eulero.

Il principale vantaggio delle rappresentazioni minime rispetto alle matrici di rotazione risiede nell'efficienza computazionale: descrivere un orientamento con soli tre parametri riduce significativamente il costo di memoria e di elaborazione nelle applicazioni pratiche.

2.5.1 Angoli ZYZ (terna corrente)

La rotazione descritta dagli angoli ZYZ viene costruita con la composizione delle seguenti rotazioni elementari:

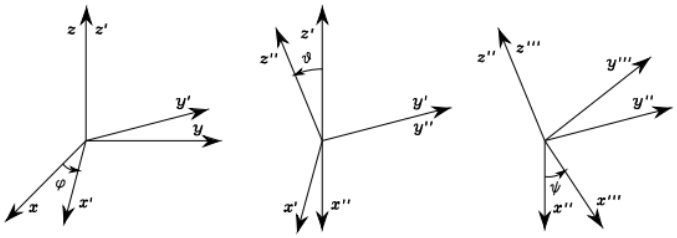


Figura 2.7: Rappresentazione di angoli di Eulero ZYZ

- Si ruota la terna origine dell'angolo φ intorno all'asse z: tale rotazione è descritta dalla matrice di rotazione $\underline{R}_z(\varphi)$
- si ruota la terna dell'angolo ϑ intorno all'asse y' corrente: tale rotazione è descritta dalla matrice di rotazione $\underline{R}_{y'}(\vartheta)$
- si ruota la terna dell'angolo ψ intorno all'asse z'' corrente: tale rotazione è descritta dalla matrice di rotazione $\underline{R}_{z''}(\psi)$

L'orientamento finale della terna, che si ricava con la composizione di rotazioni definite rispetto alla **terna corrente**, è caratterizzato dalla matrice di rotazione

ottenuta moltiplicando da sinistra verso destra le matrici rappresentative delle rotazioni elementari effettuate:

$$\underline{R}(\phi) = \underline{R}_z(\varphi)\underline{R}_{y'}(\vartheta)\underline{R}_{z''}(\psi) \tag{2.20}$$

$$R(\phi) = \begin{bmatrix} c_\varphi c_\vartheta c_\psi - s_\varphi s_\psi & -c_\varphi c_\vartheta s_\psi - s_\varphi c_\psi & c_\varphi s_\vartheta \\ s_\varphi c_\vartheta c_\psi + c_\varphi s_\psi & -s_\varphi c_\vartheta s_\psi + c_\varphi c_\psi & s_\varphi s_\vartheta \\ -s_\vartheta c_\psi & s_\vartheta s_\psi & c_\vartheta \end{bmatrix}. \tag{2.21}$$

Può essere utile risolvere il **problema inverso**, ovvero quello della determinazione degli angoli di Eulero corrispondenti a una matrice di rotazione assegnata

$$\underline{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

Dal confronto di tale espressione con quella di $\underline{R}(\phi)$ in (2.21) si ha quanto segue. Considerando gli elementi $[1,3]$ e $[2,3]$, nell'ipotesi che $r_{13} \neq 0$ ed $r_{23} \neq 0$, risulta

$$\varphi = \text{Atan2}(r_{23}, r_{13})$$

ove $\text{Atan2}(y,x)$ è la funzione arcotangente a due argomenti[1]. Quadrando e sommando gli elementi $[1,3]$ e $[2,3]$ e utilizzando l'elemento $[3,3]$, si ottiene

$$\vartheta = \text{Atan2}\left(\sqrt{r_{13}^2 + r_{23}^2}, r_{33}\right)$$

La scelta del segno positivo per il termine $\sqrt{r_{13}^2 + r_{23}^2}$ restringe a $(0,\pi)$ l'intervallo di appartenenza di ϑ. Con questa assunzione, considerando gli elementi $[3,1]$ e $[3,2]$, si ha:

$$\psi = \text{Atan2}(r_{32}, -r_{31})$$

In definitiva, la soluzione cercata risulta

$$\begin{aligned} \varphi &= \text{Atan2}(r_{23}, r_{13}) \\ \vartheta &= \text{Atan2}\left(\sqrt{r_{13}^2 + r_{23}^2}, r_{33}\right) \\ \psi &= \text{Atan2}(r_{32}, -r_{31}) \end{aligned} \tag{2.22}$$

Si può ricavare un'altra soluzione, equivalente alla (2.22) per effetti prodotti, scegliendo ϑ appartenente all'intervallo $(-\pi, 0)$. In tal caso si ottiene:

$$\varphi = \text{Atan2}(-r_{23}, -r_{13})$$

[1]La funzione $\text{Atan2}(y,x)$ calcola l'arcotangente del rapporto y/x ma utilizza il segno di ciascun argomento per determinare il quadrante di appartenenza dell'angolo risultante

$$\vartheta = \text{Atan2}\left(-\sqrt{r_{13}^2 + r_{23}^2}, r_{33}\right) \tag{2.23}$$

$$\psi = \text{Atan2}(-r_{32}, r_{31})$$

Le soluzioni (2.22), (2.23) degenerano quando $\sin_\vartheta = 0$; in questo caso è possibile determinare soltanto la somma o la differenza di φ e ψ. Infatti, se $\vartheta = 0, \pi$, le rotazioni successive di φ e ψ sono effettuate intorno ad assi di terna corrente paralleli fra di loro, fornendo così effetti di rotazione equivalenti. Queste sono dette **singolarità di rappresentazione**, sono quelle nelle quali nella risoluzione del problema inverso non riesco ad identificare univocamente la tripletta degli angoli.

2.5.2 Angoli RPY (terna fissa)

Un'altra terna di angoli di Eulero trae origine da una descrizione delle rotazioni usata frequentemente in campo (aero)nautico. Tali sono gli angoli ZYX, detti anche angoli RPY ove l'acronimo sta per *Roll-Pitch-Yaw* ovvero, rispettivamente, per il moto di rollio, beccheggio e imbardata di uno scafo/aeroplano. In questo caso la terna di angoli $\underline{\phi} = [\varphi \quad \vartheta \quad \psi]^\top$ rappresenta rotazioni definite rispetto a una terna fissa solidale al baricentro dell'aeroplano (Figura 2.8)

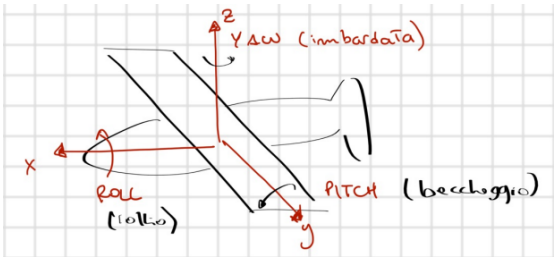


Figura 2.8: Rappresentazione di angoli RPY

- Si ruota la terna origine dell'angolo ψ intorno all'asse x (imbardata); tale rotazione è descritta dalla matrice di rotazione $\underline{R}_x(\psi)$
- si ruota la terna origine dell'angolo ϑ intorno all'asse y (beccheggio); tale rotazione è descritta dalla matrice di rotazione $\underline{R}_y(\vartheta)$
- si ruota la terna origine dell'angolo φ intorno all'asse z (rollio); tale rotazione è descritta dalla matrice di rotazione $\underline{R}_z(\varphi)$

La rotazione globale della terna, essendo ottenuta per composizione di rotazioni definite rispetto a una **terna fissa**, è caratterizzata dalla matrice ottenuta moltiplicando da destra verso sinistra le matrici rappresentative delle rotazioni elementari effettuate:[2]

[2]La sequenza ordinata di rotazioni XYZ intorno ad assi di **terna fissa** è equivalente alla sequenza ZYX intorno ad assi di **terna corrente**

$$\underline{R}(\phi) = \underline{R}_z(\varphi)\underline{R}_y(\vartheta)\underline{R}_x(\psi) \tag{2.24}$$

$$= \begin{bmatrix} c_\varphi c_\vartheta & c_\varphi s_\vartheta s_\psi - s_\varphi c_\psi & c_\varphi s_\vartheta c_\psi + s_\varphi s_\psi \\ s_\varphi c_\vartheta & s_\varphi s_\vartheta s_\psi + c_\varphi c_\psi & s_\varphi s_\vartheta c_\psi - c_\varphi s_\psi \\ -s_\vartheta & c_\vartheta s_\psi & c_\vartheta c_\psi \end{bmatrix}$$

Si può ricavare la *soluzione inversa*; assegnata la matrice di rotazione:

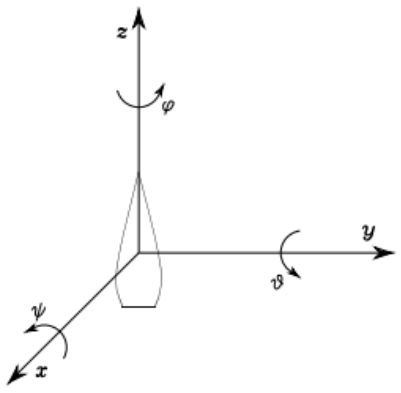
$$\underline{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix},$$

con un procedimento formalmente analogo a quello seguito in precedenza, si esegue un confronto con l'espressione di $\underline{R}(\phi)$ in (2.24). La soluzione per ϑ appartenente all'intervallo $(-\pi/2, \pi/2)$ risulta:

$$\begin{aligned} \varphi &= \text{Atan2}(r_{21}, r_{11}) \\ \vartheta &= \text{Atan2}\left(-r_{31}, \sqrt{r_{32}^2 + r_{33}^2}\right) \\ \psi &= \text{Atan2}(r_{32}, r_{33}) \end{aligned} \tag{2.25}$$

L'altra soluzione equivalente per ϑ appartenente all'intervallo $(\pi/2, 3\pi/2)$ risulta:

$$\begin{aligned} \varphi &= \text{Atan2}(-r_{21}, -r_{11}) \\ \vartheta &= \text{Atan2}\left(-r_{31}, -\sqrt{r_{32}^2 + r_{33}^2}\right) \\ \psi &= \text{Atan2}(-r_{32}, -r_{33}) \end{aligned} \tag{2.26}$$



2.6 Asse-angolo

Una rappresentazione non minima dell'orientamento può ottenersi ricorrendo a *quattro parametri* che esprimono la rotazione di un certo **angolo** θ intorno ad un **asse** r dello spazio.

Sia $r = [r_x \quad r_y \quad r_z]^\top$ il versore con cui si caratterizza un asse di rotazione rispetto alla terna di riferimento $O{-}xyz$. Per ricavare la matrice di rotazione $R(\theta, r)$ che esprime la rotazione di un angolo θ effettuata intorno all'asse r, si possono comporre rotazioni elementari intorno agli assi coordinati della terna di riferimento. L'angolo è assunto positivo se la rotazione effettuata avviene in senso antiorario intorno all'asse r.

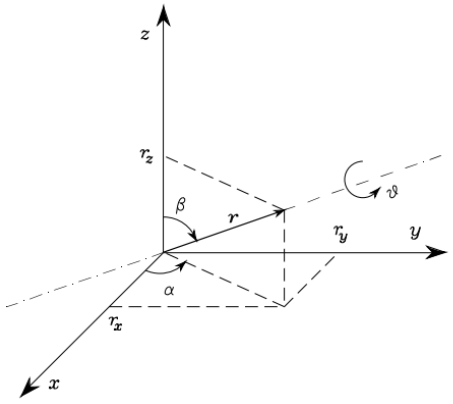


Figura 2.9: Rotazione di un angolo intorno ad un'asse

Con riferimento alla Figura 2.9, una possibile soluzione consiste nel ruotare r dell'angolo necessario a sovrapporlo all'asse z, quindi nel ruotare di θ intorno a z e infine nel ruotare dell'angolo necessario a riportare il versore nella direzione iniziale. In dettaglio la successione di rotazioni è la seguente:

- sovrapposizione di r su z, ottenuta come una successione di una rotazione di $-\alpha$ intorno a z e di $-\beta$ intorno ad y
- effettuo la rotazione voluta di θ intorno ad r
- ripristino dell'orientamento iniziale di r, ottenuto come una successione di una rotazione di β intorno ad y e di una rotazione di α intorno a z

In sintesi, la matrice di rotazione complessiva risulta:

$$R(\vartheta, r) = R_z(\alpha)R_y(\beta)R_z(\vartheta)R_y(-\beta)R_z(-\alpha) \tag{2.27}$$

Dalle componenti del versore $r = [r_x, r_y, r_z]^\top$ si possono estrarre le funzioni trascendenti necessarie al calcolo della matrice di rotazione in (2.27) in modo da eliminare la dipendenza da α e β; riconoscendo che:

$$\sin\alpha = \frac{r_y}{\sqrt{r_x^2 + r_y^2}} \qquad \cos\alpha = \frac{r_x}{\sqrt{r_x^2 + r_y^2}} \tag{2.28}$$

$$\sin\beta = \sqrt{r_x^2 + r_y^2} \qquad \cos\beta = r_z \tag{2.29}$$

si può verificare che la matrice di rotazione corrispondente a una descrizione asse e angolo è:

$$R(\vartheta, r) = \begin{bmatrix} r_x^2(1-c_\vartheta) + c_\vartheta & r_x r_y(1-c_\vartheta) - r_z s_\vartheta & r_x r_z(1-c_\vartheta) + r_y s_\vartheta \\ r_x r_y(1-c_\vartheta) + r_z s_\vartheta & r_y^2(1-c_\vartheta) + c_\vartheta & r_y r_z(1-c_\vartheta) - r_x s_\vartheta \\ r_x r_z(1-c_\vartheta) - r_y s_\vartheta & r_y r_z(1-c_\vartheta) + r_x s_\vartheta & r_z^2(1-c_\vartheta) + c_\vartheta \end{bmatrix} \tag{2.30}$$

Per la matrice di rotazione $R(\vartheta, r)$ vale la proprietà:

$$R(-\vartheta, -r) = R(\vartheta, r) \tag{2.31}$$

La (2.31) mostra come tale rappresentazione non sia univoca, poiché una rotazione di $-\vartheta$ intorno a $-r$ produce gli stessi effetti di una rotazione di ϑ intorno a r.

Considerando il *problema inverso*:

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \tag{2.32}$$

si può dimostrare:

$$\vartheta = \cos^{-1}\left(\frac{r_{11} + r_{22} + r_{33} - 1}{2}\right) \tag{2.33}$$

$$r = \frac{1}{2\sin\vartheta}\begin{bmatrix} r_{32} - r_{23} \\ r_{13} - r_{31} \\ r_{21} - r_{12} \end{bmatrix} \tag{2.34}$$

valido per $\sin\vartheta \neq 0$. Si noti che le espressioni (2.33) e (2.34) caratterizzano la rotazione in termini di quattro parametri: l'angolo e le tre componenti del versore dell'asse. Tuttavia, le tre componenti del **versore** r non sono indipendenti, ma sono vincolate dalla condizione di norma unitaria:

$$r_x^2 + r_y^2 + r_z^2 = 1 \tag{2.35}$$

Se $\sin\vartheta = 0$, le (2.33) e (2.34) perdono di significato. Per risolvere il problema inverso è necessario riferirsi alle espressioni particolari assunte dalla matrice di rotazione R e individuare le formule risolutive nei due casi $\vartheta = 0$ e $\vartheta = \pi$. Si noti che per $\vartheta = 0$ (rotazione nulla) il versore r è arbitrario (**singolarità**).

2.7 Quaternione unitario

I problemi della rappresentazione asse-angolo possono essere superati ricorrendo ad una rappresentazione a quattro parametri: il **quaternione** unitario definito come $\mathcal{Q} = \{\eta, \epsilon\}$ ove:

$$\eta = \cos\frac{\theta}{2} \tag{2.36}$$

$$\epsilon = \sin\frac{\theta}{2} r \tag{2.37}$$

Il parametro η prende il nome di parte scalare del quaternione, mentre $\epsilon = [\epsilon_x \ \epsilon_y \ \epsilon_z]^\top$ prende il nome di parte vettoriale. Su di esse vale il vincolo di norma unitaria:

$$\eta^2 + \epsilon_x^2 + \epsilon_y^2 + \epsilon_z^2 = 1$$

da cui il nome di quaternione **unitario**. È significativo notare come, a differenza di una rappresentazione asse e angolo, una rotazione di $-\vartheta$ intorno a $-r$ fornisce lo stesso quaternione di una rotazione di ϑ intorno a r.

La matrice di rotazione corrispondente a una descrizione in termini di quaternione risulta:

$$R(\eta, \epsilon) = \begin{bmatrix} 2(\eta^2 + \epsilon_x^2) - 1 & 2(\epsilon_x\epsilon_y - \eta\epsilon_z) & 2(\epsilon_x\epsilon_z + \eta\epsilon_y) \\ 2(\epsilon_x\epsilon_y + \eta\epsilon_z) & 2(\eta^2 + \epsilon_y^2) - 1 & 2(\epsilon_y\epsilon_z - \eta\epsilon_x) \\ 2(\epsilon_x\epsilon_z - \eta\epsilon_y) & 2(\epsilon_y\epsilon_z + \eta\epsilon_x) & 2(\eta^2 + \epsilon_z^2) - 1 \end{bmatrix} \tag{2.38}$$

Se è di interesse il *problema inverso* di individuare il quaternione corrispondente a una generica matrice di rotazione R:

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \tag{2.39}$$

può essere utile il seguente risultato:

$$\eta = \frac{1}{2}\sqrt{r_{11} + r_{22} + r_{33} + 1} \tag{2.40}$$

$$\epsilon = \frac{1}{2}\begin{bmatrix} \mathrm{sgn}(r_{32} - r_{23})\sqrt{r_{11} - r_{22} - r_{33} + 1} \\ \mathrm{sgn}(r_{13} - r_{31})\sqrt{r_{22} - r_{33} - r_{11} + 1} \\ \mathrm{sgn}(r_{21} - r_{12})\sqrt{r_{33} - r_{11} - r_{22} + 1} \end{bmatrix} \tag{2.41}$$

Ove si intende $\mathrm{sgn}(x) = 1$ per $x \geq 0$ e $\mathrm{sgn}(x) = -1$ per $x < 0$. Si noti che nella (2.40) è stato implicitamente assunto $\eta \geq 0$; ciò corrisponde a un angolo $\vartheta \in [-\pi, \pi]$, il che consente di descrivere qualsiasi rotazione. Inoltre, a differenza della soluzione inversa asse-angolo vista nelle (2.33) e (2.34), le espressioni dei quaternioni (2.40) e (2.41) non soffrono di singolarità.

Il quaternione estratto dalla matrice trasposta $R^{-1} = R^\top$, indicato con $\mathcal{Q}^{-1}$, può essere calcolato come:

$$\mathcal{Q}^{-1} = \{\eta, -\epsilon\} \tag{2.42}$$

Siano $\mathcal{Q}_1 = \{\eta_1, \epsilon_1\}$ e $\mathcal{Q}_2 = \{\eta_2, \epsilon_2\}$ i quaternioni corrispondenti rispettivamente alle matrici di rotazione R_1 ed R_2. Il quaternione corrispondente al prodotto delle matrici $R_1 R_2$ è dato dall'operatore prodotto tra quaternioni "$*$" :

$$\mathcal{Q}_1 * \mathcal{Q}_2 = \{\eta_1\eta_2 - \epsilon_1^\top \epsilon_2 \ , \ \eta_1\epsilon_2 + \eta_2\epsilon_1 + \epsilon_1 \times \epsilon_2\} \tag{2.43}$$

È immediato constatare che, per $\mathcal{Q}_2 = \mathcal{Q}_1^{-1}$, dalla (2.43) si ottiene il quaternione $\{1, 0\}$, che rappresenta l'elemento identità per il prodotto (**rotazione nulla**) (proprietà di **elemento identità dei quaternioni**).

2.8 Rototraslazioni (Trasformazioni omogenee)

Con riferimento alla Figura 2.10, si consideri un punto P arbitrario nello spazio. Le coordinate di P rispetto alla terna di riferimento $O_0{-}x_0y_0z_0$ siano espresse dal vettore p^0. Si consideri un'altra terna nello spazio $O_1{-}x_1y_1z_1$ e siano:

- o_1^0 il vettore che individua l'origine della terna 1 rispetto alla terna 0
- R_1^0 la matrice di rotazione della terna 1 rispetto alla terna 0
- p^1 il vettore delle coordinate di P rispetto alla terna 1

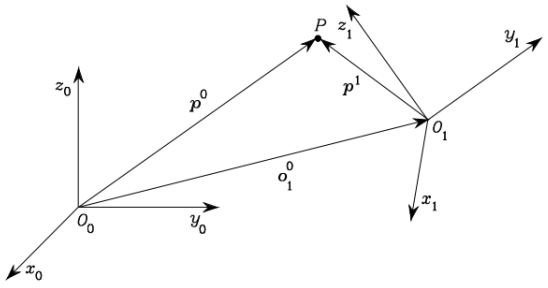


Figura 2.10: Rappresentazione di un punto P in terne differenti di coordinate

Con semplici considerazioni geometriche, si ricava che la posizione del punto P rispetto alla terna di riferimento può essere espressa come:

$$p^0 = o_1^0 + R_1^0 p^1 \tag{2.44}$$

Pertanto, la (2.44) rappresenta la **trasformazione di coordinate** (*traslazione + rotazione*) di un vettore applicato da una terna all'altra.

La trasformazione inversa può essere ricavata moltiplicando da sinistra ambo i membri della (2.44) per $R_1^{0\top}$; ricordando la proprietà di ortogonalità di una matrice di rotazione, si ha:

$$p^0 = o_1^0 + R_1^0 p^1$$
$$R_1^0 p^1 = p^0 - o_1^0$$

$$p^1 = (R_1^0)^{-1} p^0 - (R_1^0)^{-1} o_1^0$$
$$p^1 = -R_1^{0\top} o_1^0 + R_1^{0\top} p^0 \tag{2.45}$$

che, tenendo conto della relazione $R_1^{0\top} = R_0^1$, può scriversi anche come:

$$p^1 = -R_0^1 o_1^0 + R_0^1 p^0 \tag{2.46}$$

Si introduce una rappresentazione **omogenea** dei vettori. Dato un qualsiasi vettore p, la *rappresentazione omogenea* è un nuovo vettore $\tilde{p}$ costruito aggiungendo una quarta componente unitaria:

$$\tilde{p} = \begin{bmatrix} p \\ 1 \end{bmatrix} \tag{2.47}$$

Utilizzando la rappresentazione in coordinate omogenee per i vettori p^0 e p^1, la trasformazione di coordinate definita nella (2.44) può essere scritta in termini della matrice (4×4):

$$A_1^0 = \begin{bmatrix} R_1^0 & o_1^0 \\ 0^\top & 1 \end{bmatrix} \tag{2.48}$$

che prende il nome di **matrice di trasformazione omogenea**.

Come si riconosce dalla (2.48), la trasformazione di un vettore dalla terna 1 alla terna 0 è espressa da un'unica matrice contenente la matrice di **rotazione** della terna 1 rispetto alla terna 0 e il vettore di **traslazione** dall'origine della terna 0 all'origine della terna 1. Di conseguenza, la trasformazione di coordinate può riscriversi sinteticamente nella forma:

$$\tilde{p}^0 = A_1^0 \tilde{p}^1 \tag{2.49}$$

La trasformazione di coordinate tra la terna 0 e la terna 1 è descritta dalla matrice di trasformazione omogenea A_0^1, che soddisfa l'equazione:

$$\tilde{p}^1 = A_0^1 \tilde{p}^0 = (A_1^0)^{-1} \tilde{p}^0 \tag{2.50}$$

Tale matrice è espressa, in forma partizionata a blocchi, dalla:

$$A_0^1 = \begin{bmatrix} (R_1^0)^\top & -(R_1^0)^\top o_1^0 \\ 0^\top & 1 \end{bmatrix} = \begin{bmatrix} R_0^1 & -R_0^1 o_1^0 \\ 0^\top & 1 \end{bmatrix} \tag{2.51}$$

che descrive in termini di rappresentazioni omogenee il risultato già noto dalle (2.45) e (2.46). Si noti che la matrice di trasformazione omogenea non gode della proprietà di ortogonalità; pertanto:

$$A^{-1} \neq A^\top \tag{2.52}$$

Una matrice di trasformazione omogenea esprime in una forma compatta la trasformazione di coordinate tra due terne: se le terne hanno la **stessa origine**, essa si riduce essenzialmente alla **matrice di rotazione precedentemente**

definita; nel caso di terne con origini distinte, essa consente di mantenere la notazione con apice e pedice della matrice di rotazione che permette di individuare direttamente la terna corrente e la terna fissa.

In maniera analoga a quanto visto per le matrici di rotazione, si può verificare facilmente che una successione di trasformazioni consecutive di coordinate si compone per prodotto come segue:

$$\tilde{p}^0 = A_1^0 A_2^1 \dots A_n^{n-1} \tilde{p}^n \tag{2.53}$$

nella quale, in conformità alle notazioni assunte, A_i^{i-1} rappresenta la trasformazione omogenea che lega la descrizione di un punto nella terna i alla descrizione dello stesso punto nella terna $i-1$. E' quindi confermata la **proprietà di composizione**.

2.9 Cinematica diretta

Il risultato ottenuto in (2.53) è particolarmente rilevante in quanto trova applicazione nello studio della cinematica diretta dei manipolatori. Consideriamo una generica catena cinematica aperta:

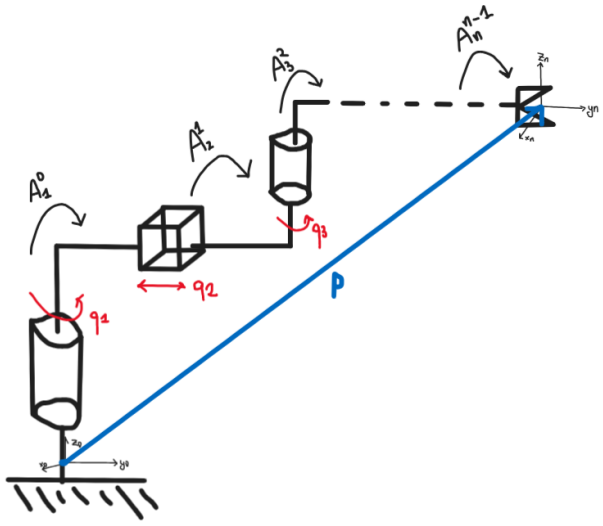


Figura 2.11: Caratterizzazione di posizione e orientamento di una catena cinematica

Lo **spazio dei giunti** è definito dalle variabili di giunto. A ciascun giunto è associato un moto elementare, che può essere una rotazione oppure una traslazione. Il **vettore delle variabili di giunto** è quindi espresso come:

$$q = [\, q_1, q_2, q_3, \dots, q_n \,]^\top$$

L'insieme di tutte le possibili configurazioni di q definisce lo spazio dei giunti.

Lo **spazio operativo** descrive invece il moto dell'organo terminale. Si assume come punto di riferimento il centro della pinza (Figura 2.11) e si descrive il suo moto nello spazio in termini di posizione e orientamento.

Per rappresentare l'orientamento sono necessarie delle terne di riferimento: si introduce una terna base $O_0{-}x_0y_0z_0$ e si associa una terna $O_n{-}x_ny_nz_n$ all'organo terminale.

La posa dell'organo terminale viene descritta tramite il **vettore posa** x, costituito dalla posizione cartesiana p e da una rappresentazione dell'orientamento ϕ della terna finale rispetto alla terna base. Generalmente, l'orientamento è espresso in forma minima mediante una delle possibili terne di angoli di Eulero:

$$x = [\, p \quad \phi \,]^\top$$

Pertanto, lo spazio operativo è costituito da tutti i possibili vettori posa, in cui:

- $p \in \mathbb{R}^{3\times 1}$ rappresenta la posizione dell'origine della terna solidale all'organo terminale
- $\phi \in \mathbb{R}^{3\times 1}$ rappresenta l'orientamento della terna n rispetto alla terna 0 in forma minima

Tra la terna base e quella dell'organo terminale vengono introdotte terne intermedie. Il problema fondamentale consiste nello stabilire una relazione tra spazio dei giunti e spazio operativo, ovvero determinare come una configurazione q influenzi la posa x dell'organo terminale. Questa relazione prende il nome di **cinematica diretta**: noto il vettore q, si vuole determinare la posa dell'end-effector.

La soluzione si ottiene attraverso una sequenza di **rototraslazioni**: a ciascun giunto viene associata una terna e il moto relativo tra terne successive è descritto mediante matrici di trasformazione omogenea. In particolare, si definiscono le matrici:

$$A_i^j$$

che rappresentano la trasformazione dalla terna j alla terna i.

Componendo tutte le trasformazioni lungo la catena cinematica, si ottiene:

$$A_n^0(q) = A_1^0(q_1)\, A_2^1(q_2) \,\ldots\, A_n^{n-1}(q_n) \tag{2.54}$$

La matrice risultante può essere scritta nella forma:

$$A_n^0(q) = \begin{bmatrix} R_n^0(q) & o_n^0(q) \\ \underline{0}^\top & 1 \end{bmatrix}$$

dove:

- $R_n^0(q)$ descrive l'orientamento dell'organo terminale rispetto alla terna base
- $o_n^0(q)$ rappresenta la posizione dell'organo terminale rispetto alla terna base

Si osserva quindi che:

$$p \equiv o_n^0(q)$$

Per determinare ϕ, si parte dalla matrice di rotazione $R_n^0(q)$ e si risolve il *problema inverso*, ricavando una parametrizzazione minima (tramite angoli di Eulero).

Nello spazio operativo, la posa è rappresentata da un vettore e non da una matrice. In forma generale:

$$p \in \mathbb{R}^{3\times 1}, \quad \phi \in \mathbb{R}^{3\times 1}$$

Il passaggio finale consiste quindi nel mappare la matrice di trasformazione 4×4 in un vettore posa 6×1. Si ottiene così la **funzione cinematica diretta**:

$$x = [\, p(q) \quad \phi(q) \,]^\top = K(q) \tag{2.55}$$

2.9.1 Convenzione di Denavit-Hartenberg

Vediamo ora come fissare le terne di riferimento. A tal fine si introduce la convenzione di Denavit-Hartenberg (DH), facendo riferimento allo schema cinematico mostrato in Figura 2.12.

Tale convenzione fornisce, per una generica catena cinematica, un insieme di regole sistematiche per definire e assegnare una terna di riferimento a ciascun giunto.

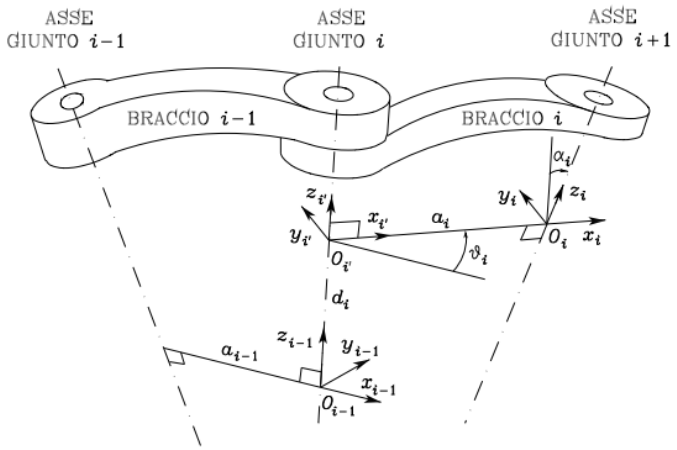


Figura 2.12: Parametri cinematici di Denavit-Hartenberg

Con riferimento alla Figura 2.12, assunto come asse del giunto i quello che connette il braccio $i-1$ al braccio i, la definizione della terna i, solidale al braccio i, avviene secondo la convenzione DH:

- A ogni giunto viene associato il proprio asse di rotazione (rappresentato tratteggiato in Figura 2.12)
- I giunti sono numerati a partire da 1, mentre le terne vengono numerate coerentemente con i giunti a cui sono associate
- Se il giunto è **rotoidale**, l'asse z_i coincide con l'asse di rotazione del giunto
- Per individuare l'asse x_{i-1}, si considerano gli assi z_{i-1} e z_i: si traccia la **normale comune**[3] tra i due assi
- Il punto di intersezione tra la normale comune e l'asse z_{i-1} individua l'origine O_{i-1}; la direzione della normale comune definisce l'asse x_{i-1}. L'asse y_{i-1} si ottiene per completamento di terna levogira
- Analogamente, per il giunto successivo, l'asse di rotazione individua z_i, mentre la normale comune tra gli assi z_i e z_{i+1} determina l'origine O_i e la direzione dell'asse x_i

Durante la costruzione delle terne, può risultare utile introdurre una terna intermedia i', detta *terna di appoggio*. Essa non ha significato fisico diretto, ma è utile per determinare in modo sistematico i parametri geometrici (parametri di Denavit-Hartenberg) necessari alla costruzione delle matrici di trasformazione.

I parametri di Denavit-Hartenberg sono definiti come segue:

- a_i: distanza tra gli assi z_{i-1} e z_i misurata lungo x_i (lunghezza del braccio)
- d_i: distanza tra le origini O_{i-1} e $O_{i'}$ misurata lungo l'asse z_{i-1}
- α_i: angolo tra gli assi z_{i-1} e z_i misurato attorno all'asse x_i (positivo in senso antiorario)
- ϑ_i: angolo tra gli assi x_{i-1} e x_i misurato attorno all'asse z_{i-1} (positivo in senso antiorario)

Tra questi quattro parametri:

- a_i e α_i sono sempre costanti, in quanto dipendono unicamente dalla geometria del manipolatore
- tra d_i e ϑ_i, uno solo è variabile, a seconda del tipo di giunto
- se il giunto i è **rotoidale**, la variabile è ϑ_i (mentre d_i è costante)

[3] La normale comune è la retta lungo la quale si realizza la minima distanza tra due assi di rotazione

- se il giunto i è **prismatico**, la variabile è d_i (mentre ϑ_i è costante)

Noti i quattro parametri, è possibile costruire la matrice di trasformazione omogenea. In particolare, la matrice A_i^{i-1} viene ottenuta come prodotto di due trasformazioni elementari che coinvolgono la terna intermedia i':

- $A_{i'}^{i-1}$ descrive la trasformazione dalla terna $i-1$ alla terna i':

  1. Si considera inizialmente una terna coincidente con la terna $i-1$
  2. Si effettua una traslazione di ampiezza d_i lungo l'asse z_{i-1} e una rotazione di angolo ϑ_i attorno allo stesso asse

  Tale trasformazione è rappresentata dalla matrice:

$$A_{i'}^{i-1} = \begin{bmatrix} \cos\vartheta_i & -\sin\vartheta_i & 0 & 0 \\ \sin\vartheta_i & \cos\vartheta_i & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- $A_i^{i'}$ descrive la trasformazione dalla terna i' alla terna i:

  1. Si effettua una traslazione di ampiezza a_i lungo l'asse $x_{i'}$
  2. Si applica una rotazione di angolo α_i attorno allo stesso asse

  La corrispondente matrice è:

$$A_i^{i'} = \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & \cos\alpha_i & -\sin\alpha_i & 0 \\ 0 & \sin\alpha_i & \cos\alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Essendo le trasformazioni espresse rispetto a terne correnti, la trasformazione complessiva si ottiene come prodotto delle due trasformazioni elementari:

$$A_i^{i-1} = A_{i'}^{i-1} A_i^{i'} = \begin{bmatrix} \cos\vartheta_i & -\sin\vartheta_i\cos\alpha_i & \sin\vartheta_i\sin\alpha_i & a_i\cos\vartheta_i \\ \sin\vartheta_i & \cos\vartheta_i\cos\alpha_i & -\cos\vartheta_i\sin\alpha_i & a_i\sin\vartheta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.56)$$

Si osservi che la matrice di trasformazione della terna i rispetto alla terna $i-1$ dipende unicamente dalla variabile del giunto i:

- ϑ_i nel caso di giunto rotoidale
- d_i nel caso di giunto prismatico

3 ROS 2

Il **Robot Operating System (ROS)** non è un vero sistema operativo, ma un **Software Development Kit (SDK)** progettato per facilitare lo sviluppo di applicazioni robotiche. Fornisce un insieme di librerie, strumenti e convenzioni che permettono di sviluppare, testare e integrare facilmente diversi moduli software all'interno di un sistema robotico. ROS mette a disposizione diversi elementi che semplificano lo sviluppo:

- **Middleware di comunicazione**: un sistema di messaggistica che permette ai moduli del robot di scambiarsi dati
- **Strumenti di sviluppo**: software utili per visualizzazione, simulazione e debugging, quali RViz, Gazebo e Rosbag
- **Pacchetti software modulari**: componenti plug-and-play che implementano funzionalità robotiche già pronte, come:
  - controllo del robot
  - pianificazione del movimento
  - percezione
  - manipolazione
  - mappatura dell'ambiente
- **Documentazione e comunità**: ROS è supportato da una vasta documentazione, tutorial e da una grande comunità di sviluppatori

ROS è ampiamente utilizzato da **ricercatori, ingegneri robotici, studenti e aziende** nel campo della robotica, in contesti che spaziano da università e laboratori di ricerca fino ad applicazioni industriali.

3.1 Ruolo di ROS nello sviluppo di sistemi robotici

Lo sviluppo di una tecnologia robotica segue generalmente un processo iterativo composto da più fasi:

1. **Individuazione del bisogno**: si parte da un problema reale o da una specifica esigenza applicativa
2. **Definizione dei requisiti tecnici**: il problema viene tradotto in specifiche tecniche per la progettazione della tecnologia
3. **Sviluppo e validazione in laboratorio**: viene realizzato un prototipo e testato in un ambiente controllato
4. **Validazione in ambiente operativo**: la tecnologia viene testata in condizioni reali per verificarne funzionalità, sicurezza e prestazioni

Dopo queste fasi, la tecnologia può essere introdotta nell'ambiente operativo attraverso:

- Ingegnerizzazione del sistema
- Certificazione
- Commercializzazione
- Addestramento degli utenti
- Valutazione delle prestazioni nel tempo

3.2 ROS 1 e ROS 2

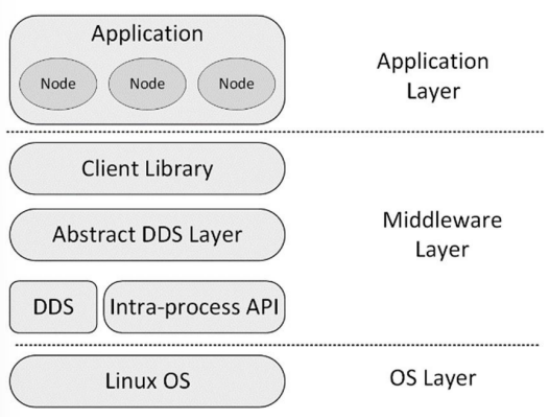
Le due principali versioni di ROS presentano alcune differenze:

- **ROS 1**: utilizzato principalmente in ricerca e prototipazione, con supporto limitato per sistemi real-time
- **ROS 2**: progettato per applicazioni industriali e di produzione, offre supporto per sistemi *real-time*, maggiore sicurezza e migliore tolleranza ai guasti

ROS è stato progettato seguendo alcuni principi fondamentali:

- **Architettura peer-to-peer**: i diversi programmi (**nodi**) comunicano tra loro tramite API ben definite utilizzando messaggi e topic
- **Sistema distribuito**: i nodi ROS possono essere eseguiti su computer diversi e comunicare attraverso la rete
- **Indipendenza dal linguaggio**: i moduli possono essere sviluppati in diversi linguaggi di programmazione, come C++ o Python
- **Open source**: gran parte dell'ecosistema ROS è libero e open source.

3.3 Architettura di ROS 2



- **Application Layer**: insieme di **nodi modulari**, o programmi eseguibili che permettono di implementare funzionalità o gestire la comunicazione con i dispositivi.
- **Middleware Layer**
  1. **DDS**: protocollo di comunicazione tra i nodi
  2. **Intra-process API**: ottimizza la comunicazione tra nodi nello stesso processo
  3. **Abstract DDS Layer**: astrazione che rende ROS 2 indipendente dalla specifica implementazione DDS
  4. **Client Library**: API per sviluppatori
- **OS Layer**: mediante Linux OS, sistema operativo su cui gira ROS 2

Il **nodo** è un **processo** che svolge una funzione specifica (*es. lettura sensore, controllo motore, pianificazione*). Può essere configurato dinamicamente. Nel middleware layer si distinguono tre strumenti di comunicazione per scambiare i dati tra i nodi (*programmi eseguibili*):

- **Topics**
- **Services**
- **Actions**

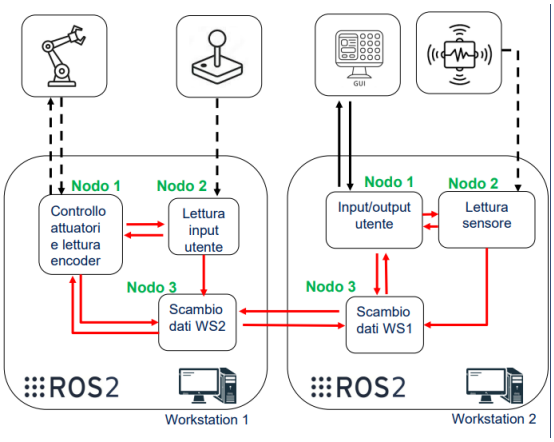


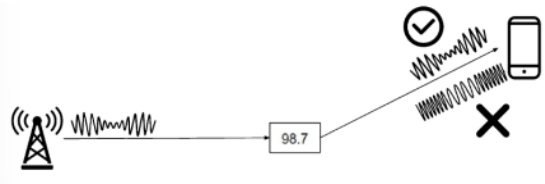
Figura 3.1: Architettura ROS con Workstation

3.3.1 Topics

Il meccanismo di comunicazione principale tra i nodi di un sistema robotico si basa sul modello **publish-subscribe**. Questo sistema permette lo scambio di informazioni in modo disaccoppiato e anonimo attraverso canali denominati **topic**.

Un **topic** può essere paragonato a una frequenza radio specifica (ad esempio, 98.7 Hz):

- **Nome:** Identifica univocamente il canale di comunicazione (es. `/car_location`).
- **Interface (Message Type):** Definisce la struttura dei dati trasmessi (es. latitudine e longitudine). Affinché la comunicazione avvenga, i partecipanti devono concordare sia sul nome che sul tipo di dato



All'interno di questo ecosistema si distinguono due ruoli:

Publisher: Il nodo che invia i dati sul topic. Agisce come un trasmettitore radio

Subscriber: Il nodo che si mette in ascolto sul topic per ricevere i dati. Agisce come un ricevitore

Il sistema è estremamente flessibile grazie ad alcune proprietà intrinseche:

- **Anonimato:** I publisher non sanno quanti o quali subscriber stiano leggendo i dati, e i subscriber non conoscono l'identità dell'entità che ha originato l'informazione
- **Cardinalità Variabile:** La comunicazione non è limitata a un rapporto 1:1. Sono supportate diverse configurazioni:
  - *One-to-many:* Un publisher e molti subscriber
  - *Many-to-one:* Più publisher sullo stesso topic e un solo subscriber
  - *Many-to-many:* Molteplici nodi che trasmettono e ricevono simultaneamente sullo stesso canale
- **Multimodalità dei Nodi:** Un singolo nodo non è limitato a un unico ruolo; può contenere al suo interno molteplici publisher e subscriber operanti su topic e interfacce differenti

Elemento	**Esempio Pratico**
Nome Topic	`/example`
Message Type	`std_msgs/String`
Contenuto	"Dati sensore posizione"

Tabella 1: Esempio di configurazione di un'interfaccia di comunicazione topic.

Sebbene il flusso dei dati sia continuo, il topic è considerato uno strumento di comunicazione **sincrono**, dove il mittente "pubblica" l'informazione senza attendere una risposta immediata dal ricevitore.

3.3.2 Tipi di messaggi in ROS 2

Per garantire l'interoperabilità tra nodi sviluppati da diversi autori, il sistema si avvale di librerie di messaggi standardizzate. Le due librerie principali sono `std_msgs` per i tipi di dato base e `geometry_msgs` per le entità spaziali.

Libreria `std_msgs` Questa libreria contiene i tipi di dato elementari (primitivi) e le loro varianti in array. È essenziale per trasmettere singoli valori di stato o liste di numeri.

- **Tipi Primitivi:** Include tipi classici come `Bool`, `String`, `Char`, e diverse precisioni per interi (`Int8`.. `Int64`, `UInt8`.. `UInt64`) e numeri in virgola mobile (`Float32`, `Float64`)
- **Tipi Array:** Permettono l'invio di sequenze di dati, come `Float32MultiArray` o `Int64MultiArray`

Libreria `geometry_msgs` Fondamentale per la robotica, definisce i messaggi per descrivere la geometria e la cinematica del robot nello spazio tridimensionale.

- **Point:** Rappresenta una posizione (x, y, z) nello spazio
- **Quaternion:** Rappresenta l'orientamento di un oggetto utilizzando i quaternioni (x, y, z, w)
- **Pose:** Combina `Point` (posizione) e `Quaternion` (orientamento) per descrivere completamente lo stato di un corpo rigido
- **Twist:** Definisce la velocità lineare e angolare (comune per il controllo dei robot mobili)
- **Transform:** Descrive la trasformazione (traslazione e rotazione) tra due sistemi di coordinate

I messaggi vengono gestiti come oggetti. È importante notare che i dati effettivi sono contenuti nel campo `.data` per i tipi semplici, o in campi specifici per i messaggi composti.

- **Messaggio Semplice (`Int64`):**

```
from std_msgs.msg import Int64
msg = Int64()
msg.data = 42
```

- **Messaggio Composto (`Pose`):** Il messaggio `Pose` è strutturato gerarchicamente:

```
from geometry_msgs.msg import Pose
msg = Pose()
# Accesso alla posizione
msg.position.x = 1.0
msg.position.y = 2.0
# Accesso all'orientamento
msg.orientation.w = 1.0
```

Molti messaggi in `geometry_msgs` possiedono una variante **Stamped** (es. `PoseStamped`). Questi aggiungono un *Header* contenente:

- **Timestamp:** Il momento esatto in cui il dato è stato generato
- **Frame_id:** Il sistema di riferimento (es. `"map"` o `"base_link"`) a cui i dati si riferiscono

3.3.3 Services

Oltre ai topic, il sistema dispone di un secondo strumento di comunicazione tra nodi denominato **Service**. A differenza del modello publish-subscribe, il service implementa un'interazione di tipo **richiesta-risposta** (request-response). Un **Service** può essere paragonato a un servizio meteo online:

- Un utente (Client) invia una richiesta contenente la propria posizione tramite un URL specifico
- Il server remoto riceve la richiesta, la elabora e restituisce le informazioni meteo locali

Come i topic, un service è definito univocamente da un **Name** (l'identificativo del servizio) e da una **Interface**, che in questo caso deve specificare obbligatoriamente sia la struttura della **richiesta** che quella della **risposta**. Il modello si basa su due entità distinte:

Service Server: Il nodo che offre il servizio. Riceve le richieste, esegue il calcolo o l'azione richiesta e invia un risultato

Service Client: Il nodo che richiede l'esecuzione del servizio. Invia i parametri necessari e attende la risposta per utilizzarla nel proprio processo interno

Il flusso operativo segue tre fasi sequenziali:

1. Il Client invia la richiesta al Server
2. Il Server processa il dato ricevuto
3. Il Server restituisce il risultato al Client, che può così proseguire la sua esecuzione

Il modello Service presenta caratteristiche strutturali differenti dai topic:

- **Cardinalità (Many-to-One):** Mentre possono esistere molteplici Client che interrogano lo stesso servizio, per ogni nome di servizio può esistere **un solo Server**. Non è possibile avere più nodi che offrono lo stesso identico service
- **Anonimato parziale:** I client non hanno bisogno di conoscere l'identità del nodo server (conoscono solo il nome del servizio/URL), e non sono a conoscenza dell'esistenza di altri client
- **Asincronicità:** a differenza dei topic, sono usati come strumenti asincroni

L'interfaccia di un servizio è definita in file con estensione `.srv`, dove una linea di separazione (`---`) distingue i campi della richiesta da quelli della risposta. Esempio di un servizio di somma aritmetica:

```
# Request
int64 a
int64 b
---
# Response
int64 sum
```

Caratteristica	**Topic**	**Service**
Modello	Publish-Subscribe	Client-Server
Flusso	Unidirezionale	Bidirezionale (Req/Res)
Cardinalità Server	Molteplici (Publisher)	Singolo (Server)
Utilizzo tipico	Flusso dati continuo	Operazioni discrete / Calcoli

Tabella 2: Confronto sintetico tra Topic e Service.

3.3.4 Action

Mentre i Topic gestiscono flussi continui e i Service operazioni discrete, le **Action** sono progettate per compiti che richiedono tempo, feedback costante e possibilità di interruzione. Sono lo standard per muovere attuatori fisici o eseguire algoritmi complessi.

Limiti dei Service nel Controllo Fisico L'utilizzo di un semplice Service per muovere un robot presenta diverse criticità:

- **Tempo di esecuzione:** Il movimento fisico non è istantaneo; il client rimarrebbe bloccato in attesa per troppo tempo
- **Assenza di Feedback:** Il client non riceve aggiornamenti sullo stato intermedio del task
- **Impossibilità di Interruzione:** Una volta inviata la richiesta, il client non può annullare l'azione in corso
- **Blocco del Server:** Il server spesso non può processare nuove richieste finché non termina quella attuale

Le **Action** risolvono questi problemi introducendo meccanismi di **feedback** e di **cancellazione (cancel)**.

Struttura dell'Interfaccia Action Un'azione è definita da un file `.action` composto da tre sezioni separate da `---`:

1. **Goal:** L'obiettivo inviato dal Client (es. la posa target da raggiungere)
2. **Result:** La risposta finale inviata dal Server al completamento (es. conferma di raggiungimento target)

3. **Feedback:** Dati periodici inviati durante l'esecuzione (es. la posa corrente del robot mentre si muove)

Un'azione non è un'entità atomica, ma un costrutto complesso che utilizza Topic e Service sottostanti per gestire il ciclo di vita del task:

- **3 Service:** Gestiscono l'invio del *Goal*, la richiesta di *Cancel* e la ricezione del *Result*
- **2 Topic:** Utilizzati per pubblicare il *Feedback* e lo *Status* del goal (macchina a stati).

Goal Status: Ogni richiesta segue una macchina a stati (es: *Accepted, Executing, Succeeded, Canceled*). Il client è avvisato automaticamente ad ogni cambio di stato

Preemption: Il client ha il potere di inviare una richiesta di cancellazione se le condizioni ambientali cambiano o per emergenza

Esempio di Flusso Operativo Consideriamo un nodo che controlla le ruote di un robot mobile:

1. L'**Action Client** invia un target (x, y)
2. L'**Action Server** accetta il goal e inizia a trasformare il target in velocità per i motori
3. Durante il movimento, il server invia costantemente la posizione attuale (**Feedback**)
4. Al termine, il server invia un messaggio di notifica (**Result**) e il robot smette di muoversi

Caratteristica	**Topic**	**Service**	**Action**
Tipo Task	Flusso dati	Calcolo rapido	Task complessi
Feedback	No	No	Sì (Continuo)
Annullabile	N/A	No	Sì
Modello	Pub/Sub	Req/Res	Goal/Result/Feedback

Tabella 3: Confronto finale tra i tre paradigmi di comunicazione.

3.4 Turtlesim

Turtlesim è un simulatore che mostra i comandi base di ROS e permette di capire come ROS controlla un robot. Da un primo terminale si avvia come:

```
ros2 run turtlesim turtlesim_node
```

Il run del nodo avviene con il comando:

```
ros2 run turtlesim turtle_teleop_key
```

esso riceve i comandi chiave per muovere la tartaruga, mediante l'uso delle frecce consente la visualizzazione del percorso. Per ottenere l'**informazione sui nodi** si possono eseguire in altri terminali altri comandi quali:

```
ros2 node list
```

```
ros2 node info /turtlesim
```

Per visualizzare i nodi e comprenderne la comunicazione si può avviare in un terminale dedicato il comando:

```
rqt_graph
```

3.4.1 Topic in Turtlesim

Per visualizzare i topic attivi nel sistema è possibile utilizzare il comando:

```
ros2 topic list
```

Per ottenere informazioni più dettagliate, inclusi i tipi di messaggio associati ai topic, si utilizza:

```
ros2 topic list -t
```

È inoltre possibile osservare in tempo reale i messaggi pubblicati su uno specifico topic tramite:

```
ros2 topic echo /turtle1/cmd_vel
```

Per visualizzare informazioni dettagliate su un topic, come il tipo di messaggio utilizzato, il numero di publisher e il numero di subscriber, si utilizza:

```
ros2 topic info /turtle1/cmd_vel
```

Il **tipo di messaggio associato ad un topic** può essere visualizzato con:

```
ros2 interface show geometry_msgs/msg/Twist
```

Il messaggio `Twist` è composto da due vettori tridimensionali che rappresentano rispettivamente la velocità lineare e quella angolare del robot. È possibile inviare comandi di movimento pubblicando direttamente un messaggio sul topic `/turtle1/cmd_vel` tramite il comando:

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist
"{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

In questo caso viene impostata una velocità lineare lungo l'asse x e una velocità angolare attorno all'asse z, producendo un movimento circolare della tartaruga nel simulatore. Se non specificata, la frequenza di pubblicazione del comando è pari a 1 Hz, generando un movimento circolare continuo.

3.4.2 Services in Turtlesim

Per visualizzare l'elenco dei service attivi nel sistema è possibile utilizzare il comando:

```
ros2 service list
```

Per conoscere il tipo di messaggio associato ad uno specifico service si utilizza:

```
ros2 service type /clear
```

La struttura dei dati utilizzati nella comunicazione tra client e server può essere visualizzata tramite:

```
ros2 interface show turtlesim/srv/Spawn
```

Il tipo di messaggio utilizzato da un service può essere ispezionato tramite:

```
ros2 interface show std_srvs/srv/Empty
```

In questo caso il service utilizza il tipo `Empty`, che non prevede dati né nella richiesta né nella risposta. Nel risultato del comando, la linea `---` separa la parte di *request* (sopra) da quella di *response* (sotto).

Un service può essere invocato direttamente dal terminale tramite il comando:

```
ros2 service call /clear std_srvs/srv/Empty
```

Nel simulatore, questo comando richiama il service `/clear`, che cancella tutte le linee disegnate dalla tartaruga e ripulisce la finestra di simulazione.

3.4.3 Actions su Turtlesim

Per analizzare le actions disponibili e la loro struttura, si utilizzano i seguenti comandi:

- **Visualizzare le azioni attive:**

  ```
  ros2 action list
  ```

- **Ottenere informazioni su un'azione specifica:** Per vedere quali nodi fungono da client e quali da server:

  ```
  ros2 action info /turtle1/rotate_absolute -t
  ```

- **Ispezionare l'interfaccia (struttura del messaggio):** L'interfaccia definisce i dati scambiati. Nel terminale, le sezioni sono separate da una linea tratteggiata (`---`):

– La **prima linea** separa il *Goal* dal *Risultato*.
– La **seconda linea** separa il *Risultato* dal *Feedback*.

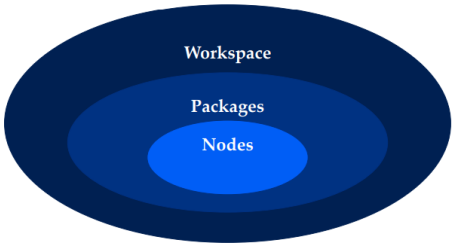
```
ros2 interface show turtlesim/action/RotateAbsolute
```

È possibile interagire con un'action server direttamente da riga di comando. Ad esempio, per ruotare la tartaruga in turtlesim:

```
ros2 action send_goal /turtle1/rotate_absolute \
turtlesim/action/RotateAbsolute "{theta: 1.0}" -f
```

3.5 Workspace

In ROS 2 il workspace è la cartella organizzativa in cui creare e fare il **build** dei packages.



Per prima cosa, è necessario creare una cartella principale e, al suo interno, la sottocartella `src`. Quest'ultima è l'unico luogo in cui l'utente deve inserire il codice sorgente delle proprie applicazioni.

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

3.5.1 Build del Workspace

A differenza di altri sistemi, un workspace ROS 2 appena creato è vuoto. Per generare l'infrastruttura necessaria, si utilizza lo strumento `colcon`. Eseguendo il build dalla radice del workspace (`ros2_ws`), verranno create automaticamente tre nuove cartelle:

- **build/**: Contiene i file intermedi generati durante la compilazione
- **install/**: Contiene i file eseguibili e gli script necessari per eseguire i pacchetti
- **log/**: Contiene i log relativi alle sessioni di build, utili per il debugging

```
colcon build
```

Dopo ogni build (o quando si apre un nuovo terminale), è fondamentale istruire il sistema sulla posizione dei nuovi pacchetti. Questo avviene tramite il comando `source`. Senza questo passaggio, ROS 2 non sarà in grado di trovare i nodi o i pacchetti appena creati.

```
source ~/ros2_ws/install/setup.bash
```

Per evitare di dover eseguire manualmente il `source` ogni volta che si apre un nuovo terminale, è consigliabile aggiungere i comandi di configurazione alla fine del file `.bashrc`. Si può procedere utilizzando un editor di testo come `gedit` o `nano`:

```
nano ~/.bashrc
```

Scorrere fino alla fine del file e aggiungere le seguenti righe (sostituendo `humble` con la propria distribuzione, es. `foxy` o `jazzy`, se diversa):

```
# Setup ambiente ROS 2 di sistema
source /opt/ros/humble/setup.bash

# Setup del workspace locale
source ~/ros2_ws/install/setup.bash
```

3.6 I Pacchetti (Packages)

Il **package** è l'unità software fondamentale in ROS 2. Serve a organizzare il codice in modo modulare e atomico, facilitando il riutilizzo e la condivisione delle funzionalità. Un pacchetto può contenere diversi elementi: nodi eseguibili, definizioni di interfacce (messaggi/servizi), file di configurazione o descrizioni hardware. A seconda dello scopo, i pacchetti si dividono principalmente in:

- **Nodi:** Contengono il codice sorgente logico (Python o C++)
- **Interfacce:** Definiscono messaggi (`.msg`), servizi (`.srv`) o azioni (`.action`) personalizzati
- **Descrizione:** Contengono file URDF o XACRO per la modellazione cinematica del robot
- **Configurazione:** Raccolgono parametri, file YAML o script di lancio (*launch files*)

I pacchetti devono essere sempre creati all'interno della cartella `src` del workspace. Per un pacchetto basato su Python, si utilizza il seguente comando:

```
cd ~/ros2_ws/src
ros2 pkg create my_py_pkg --build-type ament_python --dependencies rclpy
```

- `--build-type ament_python`: specifica che il pacchetto userà Python
- `--dependencies rclpy`: aggiunge immediatamente la libreria client di ROS 2 per Python come dipendenza

Una volta creato, il pacchetto presenta una struttura specifica necessaria per il corretto build:

- **Cartella `my_py_pkg/`:** contiene i file sorgente (`.py`) dei nodi. Deve contenere un file `__init__.py` (solitamente vuoto) per essere riconosciuta come modulo Python
- `package.xml`**:** file manifesto che elenca i metadati (nome, versione, autore) e le dipendenze del pacchetto
- `setup.py`**:** file che descrive come installare il pacchetto. Contiene il campo `entry_points`, dove vengono registrati i nodi per permetterne l'esecuzione tramite il comando `ros2 run`
- `resource/`**:** cartella utilizzata da ROS 2 per identificare la presenza del pacchetto nel sistema

Ogni volta che si aggiunge un nuovo script o si modifica la struttura del pacchetto, è necessario eseguire nuovamente il build dalla radice del workspace. Per risparmiare tempo ed evitare di ricompilare l'intero workspace, è possibile selezionare solo il pacchetto interessato:

```
cd ~/ros2_ws/
colcon build --packages-select my_py_pkg
```

Dopo ogni `colcon build`, se sono stati aggiunti nuovi eseguibili, è necessario eseguire nuovamente il `source` (o aprire un nuovo terminale se si ha configurato il `.bashrc`) affinché i nuovi nodi siano visibili.

3.7 I Nodi

Il **Nodo** è l'unità fondamentale di esecuzione in ROS 2. Un nodo è un processo (un programma eseguibile) responsabile di una specifica funzione all'interno del sistema robotico. Essi comunicano tra loro attraverso il sistema di messaggistica di ROS (Topic, Service e Action), permettendo di creare un sistema complesso e distribuito. Un singolo pacchetto può contenere più nodi correlati.

3.7.1 Sviluppo di un nodo Python

La creazione di un nodo segue un flusso di lavoro preciso suddiviso in quattro fasi:

1. Creazione del file sorgente All'interno della cartella interna del pacchetto (quella che contiene `__init__.py`), si crea il file Python e gli si assegna i permessi di esecuzione:

```
cd ~/ros2_ws/src/my_py_pkg/my_py_pkg/
touch my_first_node.py
chmod +x my_first_node.py
```

2. Scrittura del codice Il codice segue una struttura orientata agli oggetti, ereditando dalla classe `Node` di `rclpy`.

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class MyCustomNode(Node):
    def __init__(self):
        # Inizializza il nodo con il nome 'my_node_name'
        super().__init__('my_node_name')
        self.counter_ = 0
        # Crea un timer che chiama la funzione ogni 1.0 secondi
        self.timer_ = self.create_timer(1.0, self.print_hello)

    def print_hello(self):
        self.get_logger().info("Hello " + str(self.counter_))
        self.counter_ += 1

def main(args=None):
    rclpy.init(args=args)      # Inizializza la comunicazione ROS 2
    node = MyCustomNode()      # Istanzia il nodo
    rclpy.spin(node)           # Mantiene il nodo attivo (loop)
    rclpy.shutdown()           # Chiude la comunicazione in uscita

if __name__ == '__main__':
    main()
```

3. Configurazione del file setup.py Per rendere il nodo eseguibile tramite il comando `ros2 run`, bisogna registrarlo nel file `setup.py` all'interno dell'array `console_scripts`:

```
entry_points={
    'console_scripts': [
        'test_node = my_py_pkg.my_first_node:main'
    ],
},
```

Sintassi: `nome_eseguibile = nome_pacchetto.nome_file:funzione_main`

4. Build ed Esecuzione Una volta configurato, so compila il workspace e si esegue il nodo:

```
cd ~/ros2_ws/
colcon build
source ~/.bashrc
ros2 run my_py_pkg test_node
```

3.7.2 Gestione e Debug dei Nodi

Per verificare lo stato dei nodi attivi nel sistema, ROS 2 fornisce strumenti da terminale quali:

- `ros2 node list`: Elenca tutti i nodi attualmente in esecuzione
- `ros2 node info nome_nodo`: Mostra dettagli su sottoscrizioni, pubblicazioni e servizi del nodo specifico

Attenzione alle nomenclature:

- **Nome del file:** Il nome fisico dello script (es. `my_first_node.py`)
- **Nome del nodo:** Definito nel codice (`super().__init__('...')`). Deve essere **unico** nel sistema per evitare conflitti
- **Nome eseguibile:** Definito in `setup.py` e usato per il comando `ros2 run`

3.8 Launch Files

Nello sviluppo di applicazioni reali, un sistema robotico è composto da numerosi nodi che devono lavorare contemporaneamente. Avviarli manualmente uno per uno sarebbe inefficiente e prono a errori. I **Launch Files** risolvono questo problema permettendo di avviare e configurare più nodi con un unico comando. Un launch file consente di:

- Avviare più nodi contemporaneamente da un unico terminale
- Impostare parametri e argomenti per ogni nodo
- Eseguire il *remapping* dei topic (cambiare il nome dei canali di comunicazione)
- Includere altri launch file, creando una struttura gerarchica

I launch file in ROS 2 possono essere scritti in tre formati: Python (il più flessibile e consigliato), XML o YAML.

3.8.1 Creazione di un pacchetto di Launch (Bringup)

È prassi comune creare un pacchetto dedicato esclusivamente alla configurazione e al lancio del sistema, spesso chiamato `my_bringup`.

```
# 1. Creazione del pacchetto
cd ~/ros2_ws/src
ros2 pkg create my_bringup --build-type ament_cmake

# 2. Pulizia e preparazione (rimuoviamo cartelle inutilizzate)
cd my_bringup
rm -rf include/ src/
mkdir launch
```

3.8.2 Configurazione e Installazione

Per far sì che `colcon` riconosca i launch file, bisogna modificare il file `CMakeLists.txt` aggiungendo l'istruzione di installazione per la cartella `launch`:

```
install(DIRECTORY launch
  DESTINATION share/${PROJECT_NAME}
)
```

Inoltre, nel file `package.xml`, è necessario aggiungere le dipendenze di esecuzione per ogni pacchetto richiamato nel launch:

```
<exec_depend>my_py_pkg</exec_depend>
```

3.8.3 Esempio di Launch File in Python

Creiamo il file `ros2_ws/src/my_bringup/launch/action_app.launch.py`:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    ld = LaunchDescription()

    # Definizione del primo nodo
    talker_node = Node(
        package="my_py_pkg",
        executable="test_node",
        name="custom_talker_name",
        parameters=[{"use_sim_time": True}],
        remappings=[("/old_topic", "/new_topic")]
    )

    # Aggiunta delle azioni alla descrizione
```

```
    ld.add_action(talker_node)

    return ld
```

Dopo aver compilato il pacchetto, il comando per avviare il sistema è:

```
cd ~/ros2_ws/
colcon build --packages-select my_bringup
source ~/.bashrc
ros2 launch my_bringup action_app.launch.py
```

3.9 Modellazione Cinematica di un Robot

La modellazione cinematica descrive le relazioni analitiche che intercorrono tra le variabili che definiscono la configurazione interna del robot (**spazio dei giunti**) e quelle che ne determinano la posizione e l'orientamento nel mondo esterno (**spazio operativo**).

In particolare, lo studio della cinematica permette di risolvere tre problemi fondamentali:

3.9.1 Cinematica Diretta

Permette di determinare la **posa dell'end-effector** (x) a partire dalla configurazione nota dei **giunti** (q).

$$q(t) \xrightarrow{\text{Cinematica Diretta}} x(t)\,,$$

dove la posa è definita dalla posizione P_{ee}^{b} e dalla matrice di rotazione R_{ee}^{b}, spesso sintetizzate nella matrice di trasformazione omogenea:

$$T_{ee}^{b} = \begin{pmatrix} R_{ee}^{b} & P_{ee}^{b} \\ 0^{\intercal} & 1 \end{pmatrix}$$

3.9.2 Cinematica Inversa

Rappresenta il problema duale: calcolare la **configurazione dei giunti** (q) necessaria affinché l'organo terminale raggiunga una determinata **posa** (x) nello spazio.

$$x(t) \xrightarrow{\text{Cinematica Inversa}} q(t)$$

A differenza della cinematica diretta, quella inversa può presentare soluzioni multiple (diverse configurazioni per la stessa posa) o nessuna soluzione (punto fuori dal workspace).

3.9.3 Cinematica Differenziale

Mette in relazione le velocità. Permette di determinare la **velocità dell'end-effector** ($\dot{x}$) a partire dalle **velocità ai giunti** ($\dot{q}$).

$$\dot{q}(t) \xrightarrow{\text{Cinematica Differenziale}} \dot{x}(t)$$

Il legame tra queste due grandezze è espresso analiticamente attraverso la matrice **Jacobiano** (J), tale per cui $\dot{x} = J(q)\dot{q}$.

3.10 Modellazione Dinamica di un Robot

Il modello dinamico di un manipolatore descrive le relazioni matematiche tra le forze e le coppie applicate (**causa**) e il moto risultante della struttura (**effetto**). A differenza della cinematica, la dinamica tiene conto delle proprietà fisiche del robot, come masse e inerzie.

Per una simulazione accurata, il modello deve includere i seguenti parametri fisici:

- **Masse** dei singoli link
- **Centri di massa** (CoM): la posizione del baricentro di ogni link
- **Tensori d'inerzia**: descrivono come la massa è distribuita rispetto agli assi di rotazione
- **Attriti** nei giunti (statici e viscosi)
- **Caratteristiche degli attuatori**: parametri dei motori e rapporti di trasmissione

Esistono due approcci fondamentali per lo studio della dinamica:

Dinamica Diretta Viene utilizzata principalmente nella **simulazione**. Data l'applicazione di un set di coppie ai giunti (τ) e le forze di contatto con l'ambiente (h_e), il modello calcola le accelerazioni risultanti ($\ddot{q}$).

$$(\tau, h_e) \xrightarrow{\text{Dinamica Diretta}} \ddot{q}$$

Dinamica Inversa Viene utilizzata principalmente nel **controllo** del robot. Dato un moto desiderato (ovvero una traiettoria definita da accelerazioni $\ddot{q}$, velocità $\dot{q}$ e posizioni q), il modello calcola le coppie (τ) che i motori devono erogare per realizzarlo.

$$\ddot{q} \xrightarrow{\text{Dinamica Inversa}} \tau$$

3.11 URDF (Unified Robot Description Format)

L' **URDF** è un formato di file basato su XML utilizzato in ROS 2 per descrivere tutti gli elementi di un robot. Questo file permette al sistema di conoscere la geometria del robot, come sono collegati i suoi componenti e quali sono le sue proprietà fisiche. Un file URDF è composto essenzialmente da due tipi di elementi fondamentali: **Link** e **Joint**.

3.11.1 I Link (Collegamenti)

Il `<link>` rappresenta una parte rigida del robot (un "braccio" o una "base"). Per ogni link è possibile definire tre aspetti:

- **Visual:** Definisce l'aspetto estetico (forma, colore o file mesh .stl/.dae) per la visualizzazione in strumenti come **Rviz**
- **Collision:** Definisce la geometria semplificata utilizzata dal motore fisico per rilevare impatti o ostacoli
- **Inertial:** Contiene i dati dinamici analizzati precedentemente (massa e matrice di inerzia), essenziali per le simulazioni in **Gazebo**

3.11.2 I Joint (Giunti)

Il `<joint>` descrive la connessione tra due link (chiamati *parent* e *child*) e ne definisce la cinematica. I tipi di giunto più comuni sono:

- **Revolute:** Permette una rotazione attorno a un asse con limiti di angolo (giunto rotoidale)
- **Continuous:** Una rotazione continua senza limiti (es. una ruota)
- **Prismatic:** Permette uno scorrimento lineare lungo un asse
- **Fixed:** Una connessione rigida dove non è permesso alcun movimento relativo

Ecco un esempio della struttura gerarchica di un file URDF:

```
<robot name="my_robot">
  <link name="base_link">
    <visual>
      <geometry>
        <box size="0.6 0.4 0.2"/>
      </geometry>
    </visual>
  </link>

  <joint name="arm_joint" type="revolute">
    <parent link="base_link"/>
```

```
    <child link="arm_link"/>
    <origin xyz="0 0 0.2" rpy="0 0 0"/>
    <axis xyz="0 0 1"/>
    <limit effort="10.0" lower="-1.57" upper="1.57" velocity="1.0"/>
  </joint>

  <link name="arm_link"> ... </link>
</robot>
```

Poiché i file URDF per robot complessi possono diventare molto lunghi e ripetitivi, in ROS 2 si utilizza spesso XACRO, un linguaggio di macro che permette di:

- Utilizzare **costanti** e variabili (es. definire il raggio di una ruota una sola volta)
- Eseguire **operazioni matematiche** all'interno del file
- Creare **macro** (snippet di codice riutilizzabili) per evitare di duplicare il codice di parti identiche

Nello stack di navigazione e controllo, il nodo fondamentale è il `robot_state_publisher`. Questo nodo:

1. Carica il modello URDF dal file (o lo riceve come stringa)
2. Riceve lo stato attuale dei giunti (`position, velocity, effort`) dal topic `/joint_states`
3. Pubblica il topic `/robot_description` (che contiene l'intero URDF)
4. Pubblica il topic `/tf` (Transform Tree), che tiene traccia della posizione e orientamento di ogni parte del robot nello spazio in tempo reale.

3.12 Visualizzazione e Simulazione

Il framework ROS 2 offre due strumenti principali per interagire graficamente con il robot: RViz2 per l'osservazione dello stato interno e Gazebo per la simulazione del mondo fisico.

3.12.1 RViz2 (ROS Visualization)

RViz2 è uno strumento grafico che permette di visualizzare ciò che il robot pensa o vede. Non simula la fisica, ma mostra i dati provenienti dai sensori e i messaggi scambiati sui topic.

- **Cosa visualizza:** Il modello del robot (URDF), i dati dei sensori, i sistemi di coordinate (TF) e le mappe
- **Funzionamento:** Tramite il tasto `Add`, è possibile aggiungere i vari *Displays* per sottoscriversi ai topic esistenti
- **Comando:** `ros2 run rviz2 rviz2`

3.12.2 Gazebo: Il Simulatore Fisico

Gazebo è un simulatore 3D *open-source* che crea un ambiente virtuale realistico. A differenza di RViz, Gazebo possiede un **motore fisico** in grado di simulare gravità, attriti, collisioni e dinamiche.

- **SDF (Simulation Description Format):** Gazebo utilizza nativamente il formato SDF, ma ROS 2 converte automaticamente i file URDF in SDF durante il caricamento
- **Requisiti URDF per Gazebo:** Affinché un robot funzioni in Gazebo, il file URDF deve includere obbligatoriamente i tag `<inertial>` (massa e inerzia) e `<collision>`
- **Comando:** `ros2 launch gazebo_ros gazebo.launch.py`

3.12.3 Gazebo Plugins e Integrazione ROS 2

I **plugin** di Gazebo sono moduli che estendono le funzionalità del simulatore, agendo come "ponte" (*bridge*) tra il mondo simulato e ROS 2.

- **Joint State Publisher Plugin:** Pubblica lo stato dei giunti simulati sul topic `/joint_states`
- **Sensor Plugins:** Simula il comportamento di sensori reali (es. una telecamera che pubblica messaggi `Image`)
- **Joint Controller Plugin:** Permette a ROS 2 di inviare comandi di posizione o coppia ai motori simulati
- **Spawner Script:** Un nodo ROS 2 che legge la `robot_description` e instanzia il robot all'interno di Gazebo

3.12.4 Messaggi Interni Gazebo (gz msgs)

Gazebo ha un proprio sistema di messaggistica interno basato su *Protobuf*. È possibile interagire direttamente con il simulatore tramite il comando `gz topic`:

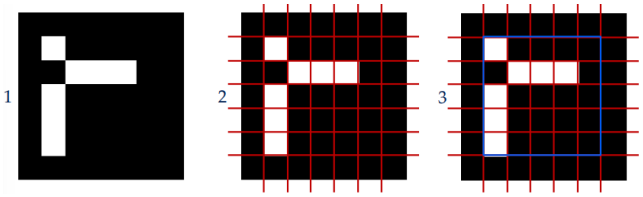
```
# Visualizzare la lista dei topic di Gazebo
gz topic -l
# Pubblicare un comando di velocità direttamente in simulazione
gz topic -t "/cmd_vel" -m gz.msgs.Twist -p "linear: {x: 0.5}, angular: {z: 0.05}"
```

3.13 ArUCO Marker

L'object detection è una tecnica che permette ad un sistema di individuare e localizzare uno o più oggetti all'interno di un'immagine o di un video. I sistemi possono essere:

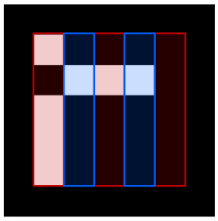
- **Markerless**: sistemi senza marker fisico, più complessi da implementare e sensibili a luce ed angolo. Richiedono ai e hanno una stima della posa complessa
- **Marker-based**: riconosce gli oggetti con marker artificiali. Sono più facili da integrare e usano librerie già esistenti. La stima della posa è più veloce dei sistemi markerless

L'ArUCO marker è un quadrato con bordo nero, con una matrice binaria interna che va a determinare l'identificatore **ID**. E' una griglia 7×7 il cui dato è visibile nella matrice interna 5×5



L'informazione all'interno della griglia è organizzata per colonne, con funzioni specifiche per il controllo degli errori e la memorizzazione dei dati:

- **Parity Bits (Colonne I, III, V):** Il loro scopo è consentire al sistema di visione di rilevare e correggere eventuali errori di lettura durante la fase di riconoscimento del codice
- **Data Bits (Colonne II, IV):** contengono i bit di dati effettivi. Essi codificano l'informazione del marker, il suo **ID univoco**, permettendo al software di distinguere un marker dall'altro all'interno di un dizionario predefinito



Poiché la griglia riserva 2 colonne da 5 quadratini l'una per i dati, si hanno a disposizione un totale di **10 data bits**. La capacità massima di codifica del sistema è quindi determinata dalla relazione:

$$N_{max} = 2^{10} = 1024 \tag{3.1}$$

Questo permette di generare identificativi (ID) univoci compresi nell'intervallo $[0, 1023]$.

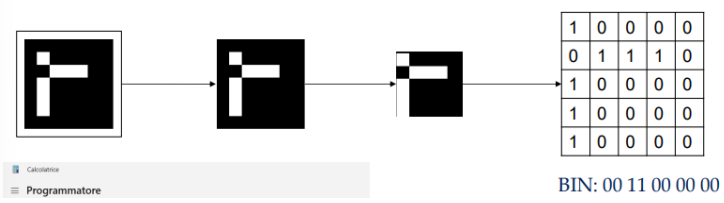
3.13.1 Algoritmo di Decodifica

Il processo di riconoscimento e decodifica di un marker ArUco avviene attraverso una sequenza di passaggi di elaborazione dell'immagine, volti a estrarre l'identificativo numerico (ID) univoco. L'algoritmo segue tre step principali:

1. **Estrazione del Marker:** Individuazione del contorno esterno dell'ArUco dall'immagine sorgente acquisita dalla telecamera
2. **Rimozione del Padding:** Eliminazione dei bordi bianchi esterni (extra padding) per isolare esclusivamente la matrice binaria interna
3. **Analisi della Griglia:** Divisione dell'immagine risultante in una griglia 5×5 per la lettura dei bit

Come analizzato precedentemente, l'informazione utile risiede esclusivamente nella II e nella IV colonna. La lettura avviene seguendo un ordine preciso:

- Dall'alto verso il basso
- Da sinistra verso destra (prima la seconda colonna, poi la quarta)



Convertendo questo valore dal sistema binario al sistema decimale, otteniamo l'identificativo del marker:

$$11000000_2 \rightarrow \text{Marker ID: } 192 \tag{3.2}$$

L'utilizzo degli ArUco marker nel campo della robotica e della computer vision è giustificato da tre caratteristiche fondamentali che ne garantiscono l'efficacia in scenari reali:

1. **Rilevamento Robusto:** Il contrasto netto tra il bordo nero e il modello binario interno permette di individuare i marcatori con successo anche in condizioni di illuminazione variabili, in presenza di distorsioni prospettiche o di occlusioni parziali
2. **Stima della Posa:** Il rilevamento dei quattro angoli del marcatore fornisce le corrispondenze geometriche necessarie per calcolare la trasformazione rigida (traslazione e rotazione) tra il sistema di coordinate della telecamera e quello del marker

3. **Rilevamento e Correzione Errore:** La natura binaria del codice interno consente l'implementazione di algoritmi di checksum per verificare l'integrità del dato e correggere eventuali bit errati

La libreria di riferimento per l'elaborazione delle immagini è **OpenCV** (Open Computer Vision), che integra un modulo specifico denominato `cv2.aruco`. Questo modulo fornisce dizionari predefiniti classificati per dimensione della griglia e numero di ID unici disponibili.

Per preparare l'ambiente di sviluppo Python, è necessario installare le librerie fondamentali tramite il gestore di pacchetti `pip`:

```
pip3 install opencv-contrib-python==4.5.4.60 transforms3d==0.4.2
```

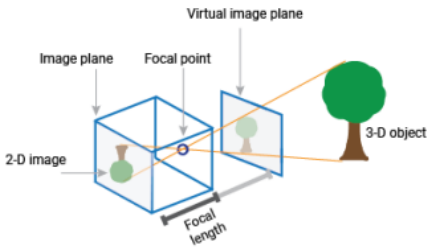
Per inizializzare un dizionario atto a generare marker con una griglia interna 4×4 e un set di 100 identificativi univoci, si utilizza la seguente istruzione:

```
aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4x4_100)
```

3.13.2 Calibrazione della Camera e Stima della Posa

Per determinare con precisione la posizione e l'orientamento di un ArUco marker rispetto alla telecamera, è necessario prima sottoporre il sensore ottico a un processo di **calibrazione**. Questo passaggio permette di correggere le imperfezioni delle lenti e mappare correttamente i punti dallo spazio 3D al piano dell'immagine 2D.

Il Modello Pinhole e Parametri Intrinseci La maggior parte delle telecamere viene modellata secondo il principio del **pinhole** (camera oscura senza lenti). La luce passa attraverso una piccola apertura e proietta un'immagine invertita sul piano focale.



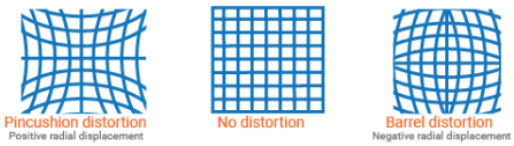
Per mappare le coordinate della camera nel piano dell'immagine si utilizza la **Camera Matrix (K)**, che contiene i parametri intrinseci:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{3.3}$$

Dove:

- f_x, f_y: Lunghezze focali lungo gli assi x e y (esprimono il fattore di ingrandimento)
- c_x, c_y: Coordinate del centro ottico rispetto al sistema di coordinate dell'immagine.

Correzione della Distorsione Le lenti reali introducono distorsioni ottiche che deformano l'immagine. I modelli più comuni sono la *Barrel distortion* (tipica dei grandangoli) e la *Pincushion distortion* (a cuscino). Tramite lo schema di calibrazione (solitamente una **scacchiera** a dimensioni note), si ottengono i **coefficienti di distorsione** necessari per "raddrizzare" l'immagine e ottenere una proiezione accurata.



In ambiente ROS 2, le informazioni di calibrazione sono pubblicate su un topic che termina solitamente in `/camera_info`. Il driver della telecamera fornisce un messaggio contenente la matrice K e il vettore di distorsione D.

Per stimare la posa, si utilizza la funzione `estimatePoseSingleMarkers`, che richiede in input:

1. I vertici dei marker individuati nell'immagine (`/image_raw`)
2. La Camera Matrix (3×3)
3. I coefficienti di distorsione

```
# Estrazione parametri in Python (ROS 2)
camera_matrix = np.array(msg.k).reshape(3, 3)
dist_coeff = np.array(msg.d)
```

Il risultato finale della stima è la posa del marker (posizione e orientamento), espressa solitamente tramite un messaggio di tipo `geometry_msgs/PoseStamped`, che permette al robot di conoscere la propria posizione rispetto all'oggetto rilevato.